

UNIDADE XII

Curso de Kotlin/Android

AUTORIA

Hugo Rodrigues Pessoni
Frederico Barbosa Relvas
Mickael Mendes Mesquita
Gustavo Novack Viana Lima



Prezado(a) Participante,

Seja bem-vindo(a) ao CEIA e Meta - AI Glasses Brasil.

Este ebook é o material de apoio dos cursos de **Kotlin/Android e de Meta SDK (Wearables Device Access Toolkit — DAT)**, que preparam as equipes para desenvolver soluções de IA embarcada com os óculos Ray-Ban Meta. Ele faz parte da Coleção Formação e Capacitação do Centro de Competências Imersivas.

O conteúdo está organizado em duas unidades. A **Unidade I — Curso de Kotlin/Android** constrói a base: a linguagem Kotlin, concorrência com coroutines e Flow, fundamentos de Android, permissões, execução em segundo plano e eficiência de energia, Edge AI (otimização e redução de modelos), visão computacional e voz no dispositivo (STT e TTS).

A **Unidade II — Curso do Meta SDK** aplica essa base ao hardware do programa: o SDK/DAT, a arquitetura da solução, o setup do ambiente, o ciclo de registro e sessão, a câmera, o áudio via Bluetooth e o desenvolvimento sem hardware com o Mock Device Kit.

Cada tópico segue a mesma estrutura: objetivos de aprendizagem, pré-requisitos, fundamentação, prática guiada, resumo e referências oficiais. **O código-fonte completo dos exemplos está no notebook companion** (`Notebook_Codigo_AKCIT_Camp.ipynb`): as caixas "► Código" ao longo do texto indicam o bloco correspondente pelo ID e pela seção. Trechos curtos aparecem também aqui no texto. Os quizzes de fixação de cada tópico estão em material de avaliação separado.

Desejamos um excelente estudo!

Unidade XII - Curso de Kotlin/Android

12.1 Introdução ao Kotlin

12.1.1 Objetivos de aprendizagem

Ao final deste tópico, você será capaz de:

- **Explicar** o que é Kotlin, de onde ela veio e por que se tornou a linguagem preferida (Kotlin-first) do Google para Android.
- **Justificar** por que o programa AI Glasses Brasil usa Android nativo em Kotlin, em vez de Java ou de frameworks cross-platform, para conversar com os óculos Ray-Ban Meta.
- **Comparar** Kotlin com Swift e com abordagens cross-platform (Flutter, React Native), entendendo o papel de cada uma.
- **Configurar** um ambiente mínimo (Android Studio + JDK) e escrever/rodar um "Olá, mundo".

Usar a sintaxe essencial (`val/var`, tipos, função `main`) o suficiente para acompanhar os próximos tópicos do módulo.

12.2 Pré-requisitos

- Nenhum tópico anterior: este é o primeiro do currículo e serve inclusive a quem não programa.
- Familiaridade básica com o uso do computador (instalar programas, abrir pastas). Nenhum conhecimento prévio de Kotlin, Java ou Android é necessário.
- Um computador **Windows 10 (64 bits)**, **macOS 12** ou **Linux 64 bits** (glibc 2.31 ou superior), com acesso à internet para baixar as ferramentas. A documentação oficial pede no mínimo **8 GB de RAM e 8 GB de disco livre** para rodar o Android Studio sozinho — mas **16 GB de RAM e 16 GB de disco** para rodá-lo junto com o emulador Android, que é o cenário deste curso. O Google recomenda 32 GB de RAM. Resolução mínima de tela: 1280 × 800.

Atenção! Máquinas Windows e Linux com processador ARM não são suportadas.

Nota 1: se a sua máquina não alcança esses números, existe a alternativa em nuvem (Android Studio on IDX) e o caminho de testar direto em um celular Android físico em vez do emulador — que, aliás, é o que você vai fazer de qualquer forma nos tópicos de IA on-device (1.6 a 1.8), porque o emulador não reflete o desempenho real de CPU/GPU.

Nota 2: e se eu quisesse programar em Swift? Swift é a linguagem irmã do Kotlin no mundo Apple (comparação no 1.1.3.5), e o IDE dela é o Xcode, hoje na linha 26. A diferença prática que interessa aqui é de pré-requisito: o Xcode só roda em macOS, em hardware Apple — desenvolver para iOS exige, obrigatoriamente, um Mac. O Android Studio, ao contrário, roda em Windows, macOS ou Linux. Como o AI Glasses Brasil é um programa Kotlin/Android, você não precisa de Mac, de Xcode nem de conta de desenvolvedor Apple em nenhum momento — e nenhuma equipe fica de fora por não ter um computador da Apple.

12.3 Conceito / fundamentação

Este é o coração do tópico: entender **o que é** o Kotlin e **por que** todo o programa é construído nele. A parte prática (instalar e rodar código) vem na próxima seção.

12.3.1 O que é Kotlin

Kotlin é uma linguagem de programação moderna, de propósito geral, criada pela **JetBrains** — a mesma empresa por trás de ferramentas de desenvolvimento amplamente usadas no mercado. Ela foi desenhada para ser concisa, segura e prática, resolvendo dores que os desenvolvedores sentiam em outras linguagens.

Se você nunca programou, pense na linguagem de programação como o idioma que usamos para dar instruções ao computador. Assim como existem idiomas humanos com regras diferentes, existem linguagens de programação com estilos diferentes. Kotlin é uma dessas linguagens — e é a que usaremos do começo ao fim deste programa.

12.3.2 Uma breve história: de 2011 a hoje

A trajetória do Kotlin ajuda a entender por que ele é levado a sério hoje:

- **2011** — a JetBrains anuncia publicamente o projeto Kotlin.
- **2016** — sai o **Kotlin 1.0**, a primeira versão estável, considerada pronta para uso em produção.
- **2017** — no evento Google I/O, o Google anuncia **suporte oficial** ao Kotlin para o desenvolvimento Android.
- **2019** — o Google torna o Kotlin a **linguagem preferida** (a estratégia "Kotlin-first"): novas APIs, exemplos e documentação passam a priorizar Kotlin.
- **Hoje** — a linguagem continua evoluindo, em ciclos previsíveis. A linha estável atual é a **2.4**, aberta pelo Kotlin 2.4.0 em junho de 2026; na data de escrita, a versão pontual mais recente dessa linha é a 2.4.10 (julho de 2026). Em julho de 2026, o Kotlin completou **15 anos** (Figura 1).

Figura 1 – Linha do tempo do Kotlin: de projeto da JetBrains a linguagem preferida do Android (2011–2026)



Fonte: autoria própria.

12.3.3 Para que serve e por que existe

Kotlin roda na **JVM** (Java Virtual Machine), a mesma máquina virtual que executa programas Java. Uma máquina virtual, aqui, é um "motor" instalado no dispositivo que sabe rodar o código compilado. Rodar na JVM traz três consequências importantes:

- **Interoperável com Java:** código Kotlin e código Java podem conviver no mesmo projeto, e o Kotlin consegue usar bibliotecas Java já existentes. Isso é

decisivo, porque o Android nasceu em Java — todo o ecossistema já pronto continua disponível.

- **Moderno e conciso:** Kotlin costuma expressar a mesma ideia com menos código que Java, o que reduz repetição e chance de erro.
- **Seguro:** o destaque é o **null safety** (segurança contra nulos), explicado a seguir.

12.3.4 Por que Kotlin, e não Java, para Android hoje

Java segue funcionando no Android, mas o Kotlin virou a escolha padrão por alguns motivos concretos:

- **Null safety.** Um dos erros mais comuns em Java é o `NullPointerException`: o programa tenta usar um valor que, na verdade, está "vazio" (`null`) e quebra em tempo de execução. Kotlin trata isso no sistema de tipos: um tipo como `String` não aceita `null`, e você precisa dizer explicitamente `String?` quando um valor pode faltar. Muitos desses erros passam a ser pegos já na compilação, antes de o app chegar ao usuário.
- **Concisão.** Menos código repetitivo (boilerplate) para tarefas comuns.
- **Coroutines.** Um mecanismo moderno para lidar com tarefas que acontecem "ao mesmo tempo" (assíncronas) de forma legível — fundamental para streaming de dados. Você verá coroutines em detalhe no **tópico 1.2**.
- **Suporte oficial do Google.** Sendo a linguagem preferida, Kotlin recebe primeiro as novidades, exemplos e ferramentas.

Na prática: no nosso programa, o **app companion Android** é quem recebe o vídeo da câmera e o áudio dos microfones dos óculos via Bluetooth e roda os modelos de IA on-device. Um app que lida com câmera, áudio e conexão sem fio tem muitos pontos onde um valor pode "faltar" (um frame que não chegou, um microfone indisponível). O null safety do Kotlin ajuda a evitar que essas situações virem travamentos no meio de uma demo do hackathon.

12.3.5 Kotlin × Swift: linguagens irmãs, plataformas diferentes

É comum confundir os papéis. A forma simples de lembrar (Tabela 1):

Tabela 1 – Comparação entre Kotlin e Swift

Papel	Kotlin	Swift
Criador	JetBrains (com o Google adotando para Android)	Apple
Plataforma principal	Android (e multiplataforma via KMP)	Ecossistema Apple: iOS, macOS
Filosofia	Moderna, concisa, segura	Moderna, concisa, segura

Fonte: autoria própria.

Ambas são linguagens modernas e seguras, com ideias parecidas — mas cada uma domina um mundo diferente. **KMP** (Kotlin Multiplatform) é a tecnologia que permite compartilhar código Kotlin entre plataformas; guarde só o nome por enquanto. Como o hardware que vamos usar é Android, Kotlin é o caminho natural.

12.3.6 Por que Android nativo em Kotlin, e não Flutter/React Native

Frameworks cross-platform (como Flutter e React Native) permitem escrever um código e rodá-lo em Android e iOS. Parece atraente, mas para este programa há motivos técnicos decisivos para ficar no nativo.

Primeiro, um esclarecimento: **o SDK da Meta não é exclusivo do Android**. O Meta Wearables Device Access Toolkit (DAT) é publicado em duas trilhas — um SDK Android (artefatos Gradle, API em Kotlin) e um SDK iOS (Swift Package Manager, Swift 6). O que ele **não** tem é trilha cross-platform:

O DAT é distribuído **apenas como SDK nativo** — Android (Kotlin) ou iOS (Swift). Não existe plugin oficial para Flutter, módulo para React Native, nem binding cross-platform.

As consequências práticas de tentar usar um framework cross-platform aqui:

- **A ponte seria sua.** Você teria que escrever e manter um platform channel (Flutter) ou native module (React Native) expondo o SDK nativo ao seu código Dart/JavaScript — trabalho que não conta como progresso no hackathon.

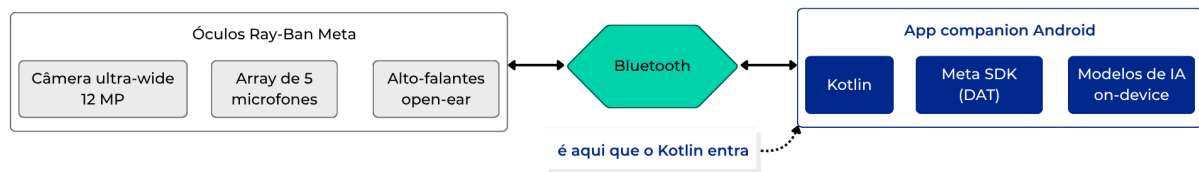
- **A API do DAT é Kotlin idiomático.** Estado exposto como `StateFlow` e `SharedFlow`, funções `suspend`, `enum` para os estados de registro e sessão (você verá tudo isso no 1.2 e no 2.4). Atravessar Flow e suspend por uma ponte é justamente a parte difícil: você reimplementaria à mão a semântica de streaming e de ciclo de sessão.
- **O gargalo de dados.** O stream de câmera entrega buffers de pixels a até 30 fps (tópico 2.5), e esses frames alimentam a inferência on-device (tópico 1.7). Fazer esses buffers atravessarem uma ponte para o lado Dart/JS e voltarem para o lado nativo do modelo acrescenta cópia e latência no trecho mais sensível do pipeline.
- **O SDK está em developer preview.** As APIs mudam entre versões — é por isso que ler o CHANGELOG é obrigatório (tópico 2.1). Uma ponte de terceiros, se existisse, andaria atrás.

Escrever em Kotlin/Android nativo dá acesso **direto e de primeira classe** ao DAT e às APIs de dispositivo (câmera, microfone, Bluetooth). Uma abordagem cross-platform precisaria de uma camada de "ponte" (bridge) para alcançar esse SDK nativo — mais complexidade, mais pontos de falha e, muitas vezes, atraso em relação aos recursos mais novos do SDK. Em um hackathon, onde tempo e estabilidade contam, o acesso direto é a escolha pragmática.

Nota 3: "nativo" quer dizer usar diretamente as ferramentas e linguagens oficiais da plataforma (aqui, Kotlin + SDK Android), sem uma camada intermediária que traduza para outra tecnologia.

Na Figura 2, veja a arquitetura da solução óculos ⇔ Bluetooth® ⇔ app companion.

Figura 2 – Arquitetura da solução: óculos, Bluetooth® e app companion Android®



Fonte: autoria própria.

12.4 Na prática

Agora vamos sair da teoria: preparar o ambiente e rodar seu primeiro programa. O objetivo aqui é mínimo — só o suficiente para você não travar nos próximos tópicos.

12.4.1 Passo 1 — O que instalar

- **Android Studio** — o ambiente de desenvolvimento (IDE) oficial para Android. É onde você escreve, roda e depura o código. **Baixe sempre a versão estável mais recente** em developer.android.com/studio; na data de escrita, a linha estável era a **Quail** (Feature Drop Quail 2 | 2026.1.2). O piso real do programa é bem mais baixo: o SDK da Meta exige **Android Studio Flamingo ou mais recente** (tópico 2.3), então qualquer versão estável atual atende com folga.
- **JDK (Java Development Kit)** — necessário porque o Kotlin roda na JVM. Boa notícia para iniciantes: o Android Studio já vem com um JDK embutido (o JetBrains Runtime), que acompanha a versão do IDE — então normalmente você não precisa instalar um JDK separado.

Nota 4: quer só experimentar Kotlin agora, sem instalar nada? Use o Kotlin Playground no navegador, em play.kotlinlang.org. É a forma mais rápida de testar os exemplos abaixo antes de montar o ambiente completo.

Nota 5: como o Android Studio é versionado, cada linha do IDE recebe um nome de animal em ordem alfabética (...Otter, Panda, Quail...), e dentro da linha saem Feature Drops numerados — Quail 1, Quail 2, Quail 3 — cada um com sua versão numérica (2026.1.1, 2026.1.2, 2026.1.3) e seus patches. Sempre há três canais em paralelo: Stable (use este), RC (quase pronto) e

Canary (experimental). Ou seja: o nome que você vê nesta página vai mudar durante o programa, e isso é normal — confira o vigente em `developer.android.com/studio` e o histórico em `developer.android.com/studio/releases`.

12.4.2 Passo 2 — Seu primeiro "Olá, mundo"

Todo programa Kotlin começa a executar por uma função especial chamada `main`. O exemplo mínimo:

► **Código 1.1-01** — A função `main` é o ponto de entrada: é por aqui que o programa começa. — código completo no notebook companion (seção 1.1.4.2).

Ao rodar (no Playground, ou pelo botão de executar do Android Studio), a saída é simplesmente:

```
Olá, mundo!
```

12.4.3 Passo 3 — `val`, `var` e tipos

Programas guardam informação em variáveis. Em Kotlin há duas formas de declarar:

► **Código 1.1-02** — código completo no notebook companion (seção 1.1.4.3).

Repare que não escrevemos o tipo (`String`, `Int`...) acima: o Kotlin usa inferência de tipo, deduzindo o tipo pelo valor. Você também pode declará-lo explicitamente:

► **Código 1.1-03** — código completo no notebook companion (seção 1.1.4.3).

Os tipos básicos que você mais verá são `String` (texto), `Int` (inteiro), `Double` (decimal) e `Boolean` (verdadeiro/falso).

E o **null safety** citado na seção anterior aparece aqui, no tipo:

```
var texto: String = "ok"
```

```
// texto = null // ERRO: String comum não aceita nulo  
  
var talvez: String? = null // OK: o "?" permite ausência de valor
```

Código 1.1-04 · também no notebook companion, seção 1.1.4.3

Na prática: prefira `val` por padrão e só use `var` quando o valor realmente precisar mudar. Em um app que processa fluxo contínuo de frames e áudio dos óculos, quanto menos estado mutável "solto", menos surpresas — é um hábito que paga dividendos quando o código cresce.

Nota 6: isso é só o mínimo para começar. Estruturas de controle, funções mais completas, classes e o modelo assíncrono (coroutines e Flow) virão nos próximos tópicos — coroutines e Flow, especificamente, no **1.2**.

12.5 Resumo / cheatsheet

- Kotlin: linguagem moderna da **JetBrains**, anunciada em **2011**, com a versão 1.0 em 2016.
- Google deu **suporte oficial em 2017** e a tornou **linguagem preferida (Kotlin-first) em 2019**.
- Roda na **JVM**, é **interoperável com Java** e se destaca por **concisão** e **null safety**.
- **Kotlin × Swift**: filosofias parecidas, plataformas diferentes — Kotlin → Android/KMP; Swift → ecossistema Apple.
- **Nativo, não cross-platform**: o SDK da Meta (DAT) é nativo Android, então Kotlin dá acesso direto ao SDK; Flutter/React Native exigiriam uma "ponte".
- **Ambiente**: Android Studio (traz um **JDK embutido**); para testar rápido, use o Kotlin Playground no navegador.
- Todo programa começa em `fun main(); println(...)` imprime texto.

- `val` = read-only (não reatribuível) · `var` = mutável; tipos básicos: `String`, `Int`, `Double`, `Boolean`; `?` habilita valores nulos.

12.6 Referências

- Kotlin Documentation — Get started / Home. Disponível em: <https://kotlinlang.org/docs/home.html> — porta de entrada da documentação oficial da linguagem, com guias e referência.
- Kotlin Documentation — Kotlin for Android. Disponível em: <https://kotlinlang.org/docs/android-overview.html> — visão geral de por que e como usar Kotlin no Android.
- Android Developers — Develop Android apps with Kotlin. Disponível em: <https://developer.android.com/kotlin> — recursos oficiais do Google sobre Kotlin no Android, incluindo a estratégia Kotlin-first.
- Android Developers — Install Android Studio. Disponível em: <https://developer.android.com/studio/install> — requisitos e passo a passo oficial de instalação do IDE.
- Android Developers — Android's Kotlin-first approach. Disponível em: <https://developer.android.com/kotlin/first> — declaração oficial e atual da política Kotlin-first do Google: por que ela existe, o que significa na prática e o que Kotlin ganha antes do Java.
- Android Developers Blog — Android's commitment to Kotlin. Disponível em: <https://android-developers.googleblog.com/2019/12/androids-commitment-to-kotlin.html> — documento de contexto histórico (dez. 2019), publicado no ano em que o Kotlin-first foi anunciado no Google I/O.
- JetBrains/Kotlin — repositório oficial. Disponível em: <https://github.com/JetBrains/kotlin> — código-fonte, releases e histórico da linguagem.

1.6.1 Para ir além



Kotlin Koans. Disponível em: <https://kotlinlang.org/docs/koans.html> — exercícios interativos guiados para praticar a sintaxe desde o zero.



What's new in Kotlin 2.4.0. Disponível em: <https://kotlinlang.org/docs/whatsnew24.html> — novidades da versão estável mais recente.

12.7 Concorrência com coroutines e flow

12.7.1 Objetivos de aprendizagem

- Ao final, você será capaz de explicar por que operações demoradas não podem bloquear a thread principal (UI) de um app Android e como coroutines evitam isso.
- Implementar funções `suspend` e iniciar coroutines com `launch` e `async` dentro de um scope adequado.
- Configurar o dispatcher correto (`Main`, `IO`, `Default`) para cada tipo de trabalho.
- Implementar e consumir um `Flow` que emite dados contínuos, como leituras de sensor ou frames de câmera.
- Aplicar boas práticas de cancelamento e ciclo de vida para evitar vazamento de coroutines.

12.7.2 Pré-requisitos

- Tópico 1.1 — Introdução ao Kotlin: funções, lambdas, classes e null safety.
- Android Studio instalado, com o mesmo ambiente de projeto Kotlin usado no 1.1.
- Dependência
`org.jetbrains.kotlinx:kotlinx-coroutines-android`. Use a

versão estável mais recente — na data de escrita, a **1.11.0** (maio de 2026).

Confira o vigente em <https://github.com/Kotlin/kotlinx.coroutines/releases>.

Atenção! Cuidado ao copiar versões da internet! A própria página de coroutines do Android Developers (developer.android.com/kotlin/coroutines), citada nas referências deste tópico, exibe no snippet de dependência a versão **1.3.9** — de 2020. O texto explicativo dela é excelente e continua válido; **o número da versão, não**. Regra geral do programa: leia o conceito na documentação, pegue o número no repositório oficial da biblioteca.

12.7.3 Conceito / fundamentação

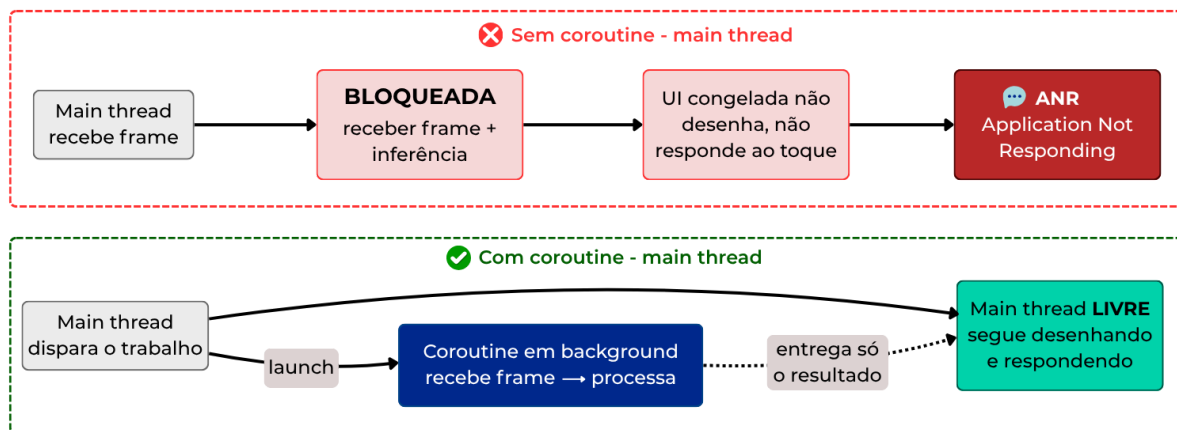
12.7.3.1 O problema: a thread principal não pode esperar

No Android existe uma única thread principal (main thread), responsável por desenhar a interface e responder aos toques do usuário. Para a UI parecer fluida, ela precisa completar cada ciclo de desenho em cerca de 16 ms (60 quadros por segundo). Se você executar nela algo demorado — esperar dados do Bluetooth, ler um arquivo, rodar a inferência de um modelo —, a tela congela. Se o bloqueio durar alguns segundos, o sistema exibe o diálogo de **ANR** (*Application Not Responding*), e o usuário provavelmente fecha o app.

Uma analogia: a main thread é o garçom do restaurante. Ele anota o pedido e o entrega à cozinha, mas não fica parado esperando o prato ficar pronto — volta a atender as mesas e só busca o prato quando a cozinha avisa. Coroutines são exatamente esse mecanismo de "avise-me quando estiver pronto" (Figura 3).

Na prática: no app companion dos óculos, tudo que importa chega de forma assíncrona: frames da câmera via Bluetooth, áudio do array de microfones, resultados da inferência dos modelos on-device. Nenhuma dessas operações pode rodar na main thread — ela deve ficar livre apenas para exibir resultados e reagir ao usuário.

Figura 3 – Main thread bloqueada × coroutine em background



Fonte: autoria própria.

12.7.3.2 Coroutines: concorrência leve

Uma **coroutine** é uma tarefa leve gerenciada pelo runtime do Kotlin, não pelo sistema operacional. Criar uma thread é caro (cada uma reserva memória própria de pilha e envolve o sistema operacional); criar uma coroutine custa quase nada — dezenas de milhares podem rodar sobre um punhado de threads. O truque: em vez de bloquear a thread enquanto espera, a coroutine **se suspende** — libera a thread para outro trabalho e retoma do ponto exato onde parou quando o resultado chega.

12.7.3.3 Funções suspend

Uma função marcada com `suspend` pode pausar sua execução sem bloquear a thread. É o bloco de construção básico:

► **Código 1.2-01** — `delay()` suspende a coroutine; a thread fica livre nesse meio-tempo — código completo no notebook companion (seção 1.2.3.3).

Regra: uma função `suspend` só pode ser chamada de outra função `suspend` ou de dentro de uma coroutine. É o compilador que garante que você nunca "espere" fora de um contexto seguro.

Atenção! `Thread.sleep(100)` e `delay(100)` parecem iguais, mas o primeiro **bloqueia** a thread (e congela a UI, se for a main) e o segundo apenas **suspende** a coroutine. Confundir os dois é um dos erros mais comuns de quem está começando.

12.7.3.4 launch e async: iniciando coroutines

Para entrar no "mundo suspend" a partir de código comum, você inicia uma coroutine com um *builder*:

- `launch` — dispara e esquece ("faça isso"); retorna um `Job`, que permite cancelar.
- `async` — dispara e devolve um resultado futuro ("calcule isso"); retorna um `Deferred<T>`, cujo valor você obtém com `await()`.

► **Código 1.2-02** — código completo no notebook companion (seção 1.2.3.4).

12.7.3.5 Scopes: toda coroutine tem dono

Coroutines seguem o princípio de *structured concurrency* (concorrência estruturada): toda coroutine nasce dentro de um `CoroutineScope`, e quando o scope é cancelado, todas as coroutines dele são canceladas juntas. Isso evita "tarefas fantasma" rodando para sempre.

- No Android, você quase nunca cria scopes na mão — usa os que já vêm amarrados ao ciclo de vida dos componentes: `viewModelScope` e `lifecycleScope` (os componentes Android e seus ciclos de vida serão vistos no tópico 1.3).

Atenção! Evite `GlobalScope`. Coroutines nele vivem enquanto o processo do app viver — se uma delas coleta frames da câmera e a tela é fechada, ela continua coletando, gastando bateria e segurando referências (vazamento de memória)

12.7.3.6 Dispatchers: em que thread o código roda

O **dispatcher** decide o conjunto de *threads* em que a coroutine executa (Tabela 2):

Tabela 2 – Tipos de dispatchers e contextos de execução

Dispatcher	Para quê	Exemplo no projeto dos óculos
<code>Dispatchers.Main</code>	Tocar na UI	Exibir a transcrição na tela

<code>Dispatchers.IO</code>	Esperas de rede, disco, <i>sockets</i>	Ler o <i>stream Bluetooth</i> , salvar áudio
<code>Dispatchers.Default</code>	Trabalho pesado de CPU	Decodificar frame, pré-processar imagem para o modelo

Fonte: autoria própria.

Para trocar de dispatcher no meio de uma função, use `withContext`:

► **Código 1.2-03** — código completo no notebook companion (seção 1.2.3.6).

Na prática: um pipeline típico do hackathon usa os três: `IO` para receber os bytes do frame pelo Bluetooth, `Default` para pré-processar e rodar a inferência (tópicos 1.6 e 1.7) e `Main` apenas para mostrar o resultado.

12.7.3.7 Flow: fluxos contínuos de dados

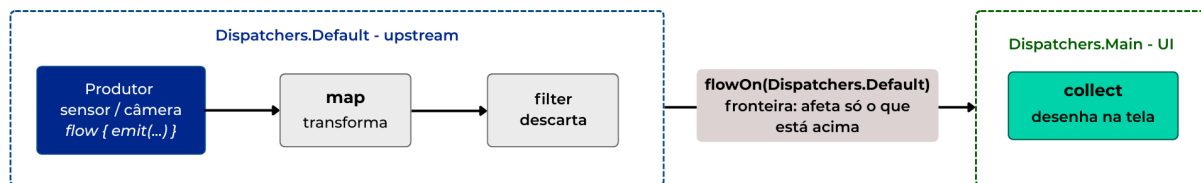
Uma função `suspend` devolve **um** valor e termina. Mas câmera e microfone não entregam um valor: entregam uma sequência que não acaba — dezenas de frames por segundo, blocos contínuos de áudio, resultados parciais de transcrição (tópico 1.8). Para isso existe o `Flow<T>`: um fluxo assíncrono que emite vários valores ao longo do tempo.

Dois comportamentos definem um Flow:

- Ele é **frio** (*cold*): o código produtor só executa quando alguém chama `collect`. Sem coletor, nada acontece.
- Ele é **sequencial e suspenso**: cada `emit` respeita o ritmo do coletor, sem bloquear threads.

Entre produtor e coletor você encadeia **operadores**, no mesmo espírito das operações de coleção vistas no 1.1: `map` transforma, `filter` descarta, `flowOn` define o dispatcher do trecho produtor (Figura 4).

Figura 4 – Pipeline de Flow e a fronteira do flowOn



Fonte: autoria própria.

Nota 7: além do Flow frio, existem variações "quentes", como `StateFlow`, que guarda o último valor emitido — útil para representar estado de tela. Aqui o foco é o Flow básico; veja as referências para aprofundar.

12.7.3.8 Cancelamento e ciclo de vida: evitando vazamentos

O cancelamento em coroutines é **cooperativo e em cascata**:

- Cancelar um `Job` (ou o scope inteiro) cancela todos os filhos.
- As funções `suspend` da biblioteca (`delay`, `emit`, `withContext`) verificam o cancelamento automaticamente; em loops de cálculo puro, verifique você mesmo com `isActive` ou `ensureActive()`.
- O cancelamento é sinalizado por uma `CancellationException` — nunca a "engula" com um `catch` genérico sem relançá-la.
- Use `try/finally` (ou `awaitClose`, em `callbackFlow`) para liberar recursos: fechar o stream da câmera, parar o microfone.

O vazamento clássico: iniciar a coleta de um Flow em um scope que vive mais que a tela. O usuário sai da tela, mas a coroutine continua coletando frames — desperdiçando CPU, bateria e banda Bluetooth. A regra de ouro: colete fluxos ligados à UI em scopes cientes do ciclo de vida (`lifecycleScope` + `repeatOnLifecycle`, detalhados no tópico 1.3).

12.7.4 Na prática

Vamos construir, passo a passo, um Flow que simula um sensor emitindo leituras contínuas — o mesmo padrão que você usará com os frames dos óculos.

12.7.4.1 Passo 0 — Dependência

► **Código 1.2-04** – `build.gradle.kts` (módulo app) — código completo no notebook companion (seção 1.2.4.1).

12.7.4.2 Passo 1 — O sensor simulado

► **Código 1.2-05** – código completo no notebook companion (seção 1.2.4.2).

Repare: `flow { }` cria o Flow, `emit` publica um valor e `delay` marca o ritmo sem bloquear. Como o Flow é frio, nada disso executa ainda.

12.7.4.3 Passo 2 — Coletando e transformando

► **Código 1.2-06** – código completo no notebook companion (seção 1.2.4.3).

Execute como um programa Kotlin comum e você verá as leituras surgindo em tempo real. O `take(20)` cancela o produtor automaticamente após a 20ª emissão — o `while (true)` não roda para sempre.

Atenção! Erro comum: criar o Flow e "não acontecer nada". Quase sempre a causa é não ter chamado `collect` (ou tê-lo chamado fora de uma coroutine). Flow frio sem coletor é só a receita, não a comida.

12.7.4.4 Passo 3 — Tirando o trabalho pesado da main thread

Se o `map` fizesse algo caro (decodificar imagem, pré-processar para a inferência), você moveria o trecho produtor para `Default` com `flowOn`:

► **Código 1.2-07** – código completo no notebook companion (seção 1.2.4.4).

Regra do `flowOn`: ele afeta o que está **acima** dele na cadeia (*upstream*), nunca o `collect`.

12.7.4.5 Passo 4 — Coletando com segurança na UI do Android

Em um app real, amarre a coleta ao ciclo de vida da tela para que ela comece e pare automaticamente:

► **Código 1.2-08** – Dentro de uma Activity (Activity e lifecycle são detalhados no tópico 1.3) — código completo no notebook companion (seção 1.2.4.5).

Quando o usuário sai da tela, a coleta é cancelada; quando volta, recomeça. Zero vazamento.

12.7.4.6 Passo 5 — E quando a fonte usa callbacks?

SDKs de hardware (câmera, microfone) costumam entregar dados por **callback**, não por Flow. O adaptador padrão é `callbackFlow`:

► **Código 1.2-09** – código completo no notebook companion (seção 1.2.4.6).

O `awaitClose` é obrigatório: é ele que garante que, quando o coletor cancelar (tela fechada), o listener será removido e o hardware liberado.

Na prática: este é exatamente o formato do streaming dos óculos. Os frames da câmera e os blocos de áudio chegarão ao app companion por APIs assíncronas (tópicos 2.5 e 2.6), e você os tratará como Flows — o pipeline `map/filter/flowOn` que você montou aqui para o sensor simulado é o mesmo que alimentará a inferência on-device.

12.7.4.7 Erros comuns

Os erros mais comuns estão descritos na Tabela 3.

Tabela 3 – Erros comuns no uso de coroutines e Flows

Sintoma	Causa provável	Correção
UI congela ou ANR	Trabalho pesado ou <code>Thread.sleep</code> na main thread	<code>withContext(Dispatchers.Default/IO)</code> ou <code>flowOn</code>
"Suspend function should be called only from a coroutine..."	Chamar <code>suspend</code> fora de coroutine	Envolver em <code>scope.launch { }</code>

Flow "não emite"	Falta de <code>collect</code> (Flow é frio)	Coletar dentro de uma coroutine
Coleta contínua com a tela fechada	Scope errado (<code>GlobalScope</code> ou scope manual)	<code>lifecycleScope</code> + <code>repeatOnLifecycle</code>

Fonte: autoria própria.

12.7.5 Resumo / cheatsheet

- A main thread só desenha UI e reage ao usuário; qualquer espera ou cálculo pesado vai para coroutines.
- `suspend` pausa sem bloquear; `delay` ≠ `Thread.sleep`.
- `launch` = disparar sem resultado (retorna `Job`); `async/await` = disparar com resultado (`Deferred<T>`).
- Toda coroutine vive em um scope; no Android, prefira `viewModelScope/lifecycleScope`; evite `GlobalScope`.
- Dispatchers: `Main` = UI · `IO` = rede/disco/Bluetooth · `Default` = CPU pesada.
- `Flow` é frio: só produz quando coletado; `map/filter` transformam o fluxo no caminho.
- `flowOn` muda o dispatcher do trecho acima dele (upstream); o `collect` roda no contexto do chamador.
- Cancelamento é cooperativo e em cascata: cancele pelo scope e use `awaitClose/finally` para liberar câmera e microfone.

12.7.6 Referências

- COROUTINES / KOTLIN DOCS. Disponível em: <https://kotlinlang.org/docs/coroutines-overview.html>. Visão geral oficial de coroutines: suspend, builders e structured concurrency.
- ASYNCHRONOUS FLOW / KOTLIN DOCS. Disponível em: <https://kotlinlang.org/docs/flow.html>. Guia completo de Flow: cold streams, operadores, flowOn e cancelamento.

- KOTLIN COROUTINES ON ANDROID / ANDROID DEVELOPERS. Disponível em: <https://developer.android.com/kotlin/coroutines>. Coroutines no contexto Android: scopes, dispatchers e integração com o framework.
- KOTLIN FLOWS ON ANDROID / ANDROID DEVELOPERS. Disponível em: <https://developer.android.com/kotlin/flow>. Flow em apps Android, incluindo coleta segura com o ciclo de vida e StateFlow.
- BEST PRACTICES FOR COROUTINES IN ANDROID / ANDROID DEVELOPERS. Disponível em: <https://developer.android.com/kotlin/coroutines/coroutines-best-practices>. Regras práticas: injetar dispatchers, evitar GlobalScope e tratar cancelamento.
- KOTLINX.COROUTINES / GITHUB (JETBRAINS). Disponível em: <https://github.com/Kotlin/kotlinx.coroutines>. Repositório oficial, com changelog e a versão estável atual da biblioteca.

12.7.6.1 Para ir além



USE KOTLIN COROUTINES WITH LIFECYCLE-AWARE COMPONENTS / ANDROID DEVELOPERS. Disponível em: <https://developer.android.com/topic/libraries/architecture/coroutines>. Detalha lifecycleScope, viewModelScope e repeatOnLifecycle.



COROUTINES GUIDE / KOTLIN DOCS. Disponível em: <https://kotlinlang.org/docs/coroutines-guide.html>. Índice do guia aprofundado, do básico a channels e tratamento de exceções.

12.8 Fundamentos de Android

12.8.1 Objetivos de aprendizagem

Ao final, você será capaz de:

- Explicar a estrutura de um projeto Android com Gradle (Kotlin DSL), o papel dos módulos e do `AndroidManifest.xml`.

- Descrever o ciclo de vida de uma Activity e justificar por que ele importa quando o app mantém uma sessão com os óculos e recebe stream de câmera.
- Implementar um `ViewModel` que expõe o estado da tela via `StateFlow`, sobrevivendo a mudanças de configuração.
- Construir uma UI mínima em Jetpack Compose com um botão e um texto que reagem a estado.
- Comparar onde cada tipo de estado deve viver (Activity vs. ViewModel) no app companion dos óculos.

12.8.2 Pré-requisitos

- **Tópico 1.1 — Introdução ao Kotlin:** sintaxe, `data class`, lambdas e referências de função.
- **Tópico 1.2 — Coroutines e Flow:** `viewModelScope`, `launch`, o que é um `Flow` e como coletá-lo.
- **Android Studio** instalado, com um emulador ou dispositivo físico Android configurado.

12.8.3 Conceito / fundamentação

12.8.3.1 Por que você precisa de um app Android

Os Ray-Ban Meta não têm display nem rodam seus modelos: eles capturam (câmera ultra-wide de 12 MP, array de 5 microfones) e reproduzem áudio (alto-falantes open-ear). Todo o "cérebro" da sua solução — os modelos de IA on-device, a lógica do agente, a UI de configuração — roda no **app companion Android**, conectado aos óculos via Bluetooth. Este tópico monta a estrutura mínima desse app: um projeto que compila, uma tela que reage a estado e uma base sólida para hospedar o modelo e o agente nos tópicos seguintes.

12.8.3.2 Anatomia de um projeto Android (Gradle + Kotlin DSL)

Um projeto Android é, na essência, um conjunto de **módulos** orquestrados pelo **Gradle**, o sistema de build. Cada módulo é uma unidade compilável: o módulo

`:app` gera o APK; módulos de biblioteca (ex.: `:core`, `:ml`) geram código reutilizável. Para o hackathon, um único módulo `:app` é suficiente — só crie módulos extras se o time crescer e precisar isolar responsabilidades.

Os scripts de build são escritos em **Kotlin DSL** (arquivos `.gradle.kts`) — a mesma linguagem do seu código, com autocompletar e checagem de tipos, em vez do Groovy legado (Tabela 4).

Tabela 4 – Principais arquivos de configuração de um projeto Android

Arquivo	Papel
<code>settings.gradle.kts</code>	Declara quais módulos existem no projeto e de onde baixar dependências (repositórios).
<code>build.gradle.kts</code> (raiz)	Configurações comuns a todos os módulos; declara plugins sem aplicá-los.
<code>app/build.gradle.kts</code>	Configura o módulo <code>:app</code> : <code>namespace</code> , <code>minSdk/targetSdk</code> , e o bloco <code>dependencies</code> .
<code>gradle/libs.versions.toml</code>	<i>Version catalog</i> : centraliza nomes e versões de bibliotecas em um só lugar.
<code>app/src/main/AndroidManifest.xml</code>	"Certidão de nascimento" do app: componentes, permissões e requisitos de <i>hardware</i> .

Fonte: autoria própria.

Um recorte típico do `app/build.gradle.kts`:

► **Código 1.3-01** – código completo no notebook companion (seção 1.3.3.2).

Nota 8: os projetos novos criados pelo Android Studio já vêm com Kotlin DSL e version catalog por padrão — você não precisa migrar nada.

12.8.3.3 AndroidManifest: o contrato com o sistema

O `AndroidManifest.xml` declara ao Android **o que seu app é e do que precisa**: quais componentes existem (Activities, Services...), quais permissões o app pode pedir e qual Activity abre quando o usuário toca no ícone. É nele que, no tópico 1.4, você declarará permissões como microfone e Bluetooth.

► **Código 1.3-02** – código completo no notebook companion (seção 1.3.3.3).

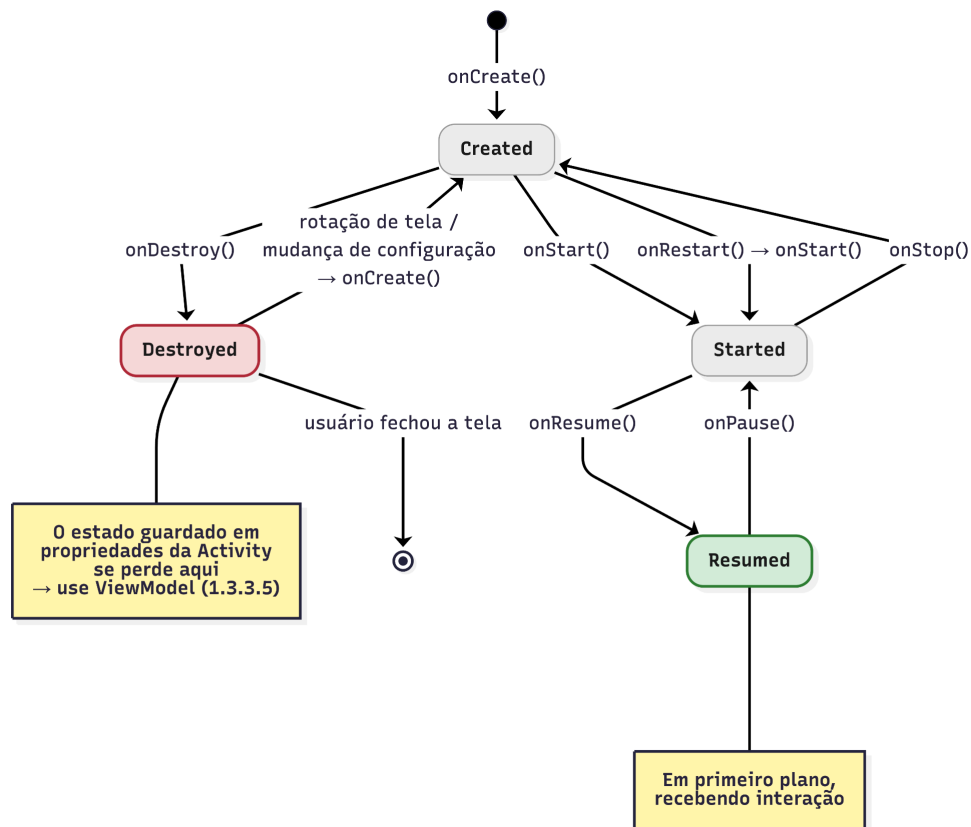
Atenção: Declarar uma permissão no `Manifest` não a concede — permissões sensíveis (câmera, microfone) exigem pedido em *runtime*, assunto do tópico 1.4.

12.8.3.4 Ciclo de vida da Activity

Uma **Activity** é uma tela do app, e o Android cria e destrói Activities conforme o usuário navega, atende uma ligação ou gira o aparelho. O sistema avisa cada transição chamando *callbacks* que você pode sobrescrever:

- `onCreate()` — a Activity nasce; configure a UI aqui (`setContent` no `Compose`).
- `onStart()` — a tela ficou **visível**.
- `onResume()` — a tela está em **primeiro plano**, recebendo interação.
- `onPause()` — perdeu o primeiro plano (ex.: diálogo por cima); pode ainda estar visível.
- `onStop()` — deixou de ser visível (usuário foi para outro app).
- `onDestroy()` — a Activity morre: o usuário fechou a tela **ou** o sistema a recriou (Figura 5).

Figura 5 – Ciclo de vida da Activity



Fonte: autoria própria.

O detalhe que mais derruba iniciantes: uma **mudança de configuração** (girar a tela, trocar idioma, redimensionar em multi-window) **destrói e recria a Activity**. Qualquer estado guardado em propriedades da Activity — um contador, um texto, uma conexão — se perde.

Na prática: no app companion, o ciclo de vida dita o consumo de recursos caros. O preview da câmera dos óculos chegando por Bluetooth (tópico 2.5) não deve continuar sendo renderizado com o app invisível: pause-o em `onStop()` e retome em `onStart()`. Ignorar isso drena a bateria dos óculos e do celular — e bateria é um dos recursos mais escassos do hackathon. O que pode (e o que não pode) continuar em segundo plano é assunto do tópico 1.5.

12.8.3.5 ViewModel: estado que sobrevive à recriação

Se a Activity morre na rotação, onde guardar o estado da tela? No `ViewModel`, componente do Jetpack criado exatamente para isso: ele é retido pelo sistema durante mudanças de configuração e só é destruído quando a tela sai

definitivamente da navegação. A Activity recriada se reconecta ao **mesmo** ViewModel e encontra o estado intacto.

O ViewModel também é o dono do `viewModelScope` (visto no 1.2): coroutines lançadas nele são canceladas automaticamente quando o ViewModel morre — sem vazamento.

Pense na divisão assim: a Activity é a **moldura** (nasce e morre ao sabor do sistema); o ViewModel é o **quadro** (o conteúdo que persiste enquanto a tela existir logicamente).

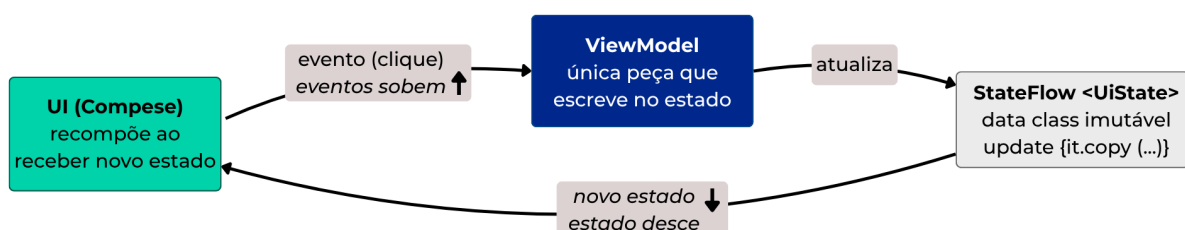
12.8.3.6 StateFlow: a fonte única do estado da tela

O ViewModel guarda o estado, mas a UI precisa **reagir** quando ele muda. A ponte é o `StateFlow` (visto no 1.2): um Flow que sempre tem um valor atual e emite cada novo valor aos coletores. O padrão consagrado:

1. O estado da tela é uma `data class` imutável (`UiState`).
2. O ViewModel mantém um `MutableStateFlow` **privado** — só ele escreve.
3. A UI recebe uma versão **somente leitura** (`StateFlow`) e apenas observa.
4. As interações do usuário viram **chamadas de função** no ViewModel, nunca escrita direta no estado.

Esse desenho é o *unidirectional data flow* (UDF, fluxo de dados unidirecional): estado desce, eventos sobem (Figura 6).

Figura 6 – Fluxo de dados unidirecional (UDF)



Fonte: autoria própria.

12.8.3.7 Jetpack Compose: UI declarativa mínima

Jetpack Compose é o toolkit moderno de UI do Android. Em vez de manipular views imperativamente ("pegue o `TextView` e troque o texto"), você **descreve** a tela como função do estado: `UI = f(state)`. Quando o estado muda, o Compose reexecuta as funções afetadas — a **recomposição** — e a tela se atualiza sozinha.

Funções de UI são marcadas com `@Composable` e montadas por composição: `Column`, `Text`, `Button` etc. Para o app companion, isso basta: a experiência do usuário final está na voz e no áudio dos óculos; a tela do celular é só painel de controle — conectar, ver status, ajustar.

Na prática: resista à tentação de investir tempo de hackathon em UI elaborada. Os jurados verão a demo pelos óculos (voz e áudio), não pelo celular. Uma tela com status da conexão e um botão já cumpre o papel.

12.8.4 Na prática

Vamos construir o esqueleto do app companion: uma tela com um texto de status e um botão "Conectar" que simula o handshake com os óculos. A conexão real via Meta SDK será vista no tópico 2.4 — aqui o foco é a estrutura.

12.8.4.1 Passo 1 — Criar o projeto

No Android Studio: **New Project** → **Empty Activity**. Apesar do nome genérico, este é o template Compose — ele já vem com as dependências do Compose e o Material Design configurados, e a documentação oficial o indica como ponto de partida recomendado para qualquer projeto novo. Nomeie **Companion**, pacote `com.exemplo.companion`, linguagem **Kotlin** (é a única opção para Compose). Em **Minimum SDK**, escolha **API 29 (Android 10)** — é o piso exigido pelo SDK da Meta, que você vai adicionar no tópico 2.3; escolher menos agora significa refazer depois. O wizard gera a estrutura Gradle Kotlin DSL + version catalog da seção anterior.

Nota 9: se você encontrar tutoriais mandando escolher **"Empty Compose Activity"**, é o nome antigo do mesmo template — foi renomeado para "Empty Activity". Os templates com "Views" no nome (ex.: "Empty Views Activity") são os de UI em XML, o caminho legado, que não usaremos.

Atenção, usuários de macOS! A documentação oficial recomenda salvar o projeto em `/Users/seu_usuario/AndroidStudioProjects`. Pastas como Documentos, Desktop e Downloads são protegidas pelo mecanismo TCC do macOS, e os arquivos do projeto podem não aparecer corretamente no Android Studio sem conceder permissão especial ao IDE.

12.8.4.2 Passo 2 — Conferir dependências

O template já inclui Compose e Activity. Adicione (via version catalog) o suporte a ViewModel no Compose e à coleta lifecycle-aware:

► **Código 1.3-03** — `app/build.gradle.kts` — bloco dependencies — código completo no notebook companion (seção 1.3.4.2).

12.8.4.3 Passo 3 — Modelar o estado da tela

► **Código 1.3-04** — `UiState.kt` — o estado da tela em uma data class imutável — código completo no notebook companion (seção 1.3.4.3).

12.8.4.4 Passo 4 — Escrever o ViewModel

► **Código 1.3-05** — `MainViewModel.kt` — código completo no notebook companion (seção 1.3.4.4).

Nota 10: `update { }` aplica a mudança de forma atômica: recebe o estado atual e devolve uma **cópia** modificada (`copy` da data class). Nunca mute o objeto de estado — crie um novo.

12.8.4.5 Passo 5 — Construir a UI em Compose

► **Código 1.3-06** — `MainScreen.kt` — código completo no notebook companion (seção 1.3.4.5).

12.8.4.6 Passo 6 — Ligar a Activity

► **Código 1.3-07** – MainActivity.kt — código completo no notebook companion (seção 1.3.4.6).

12.8.4.7 Passo 7 — Rodar e provar o ciclo de vida

Execute no emulador. Toque em "Conectar aos óculos", espere o status virar "Conectado" e **gire a tela**: a Activity é destruída e recriada, mas o texto continua "Conectado" — o `UiState` mora no ViewModel, que sobreviveu. Guarde esse teste: é a prova de que seu estado está no lugar certo.

Na prática: esse esqueleto é literalmente a base do seu app do hackathon. Nos tópicos do Módulo 2, o `delay` simulado dará lugar à sessão real com os óculos (2.4) — e é o ViewModel que vai segurar essa sessão, imune às rotações de tela durante a demo.

12.8.4.8 Erro comum 1 — Expor o MutableStateFlow

```
val uiState = MutableStateFlow(UiState()) // público e mutável: ruim!
```

Código 1.3-08 · também no notebook companion, seção 1.3.4.8

Qualquer Composable poderia escrever no estado, quebrando o UDF e tornando bugs impossíveis de rastrear. **Correção:** campo `private val _uiState` + exposição via `asStateFlow()`, como no Passo 4.

12.8.4.9 Erro comum 2 — Coletar sem consciência de lifecycle

Usar `collectAsState()` (sem `WithLifecycle`) mantém a coleta ativa mesmo com o app em segundo plano. Com um contador de status é inofensivo; com frames de câmera chegando dos óculos, é desperdício de CPU e bateria. **Correção:** prefira `collectAsStateWithLifecycle()`, que segue as regras de `onStart/onStop` automaticamente.

Atenção! Se `collectAsStateWithLifecycle` não for resolvido, falta a dependência `androidx.lifecycle:lifecycle-runtime-compose` do Passo 2 — o erro de import confunde porque `collectAsState` (de outro pacote) continua disponível.

12.8.5 Resumo / cheatsheet

- Módulo `:app` + Gradle Kotlin DSL (`.gradle.kts`) + version catalog (`libs.versions.toml`): estrutura padrão de projeto; dependências vivem em `app/build.gradle.kts`.
- `AndroidManifest.xml` declara componentes, permissões e a Activity de entrada (`MAIN/LAUNCHER`) — declarar permissão não é concedê-la (tópico 1.4).
- Ciclo de vida: `onCreate` → `onStart` → `onResume` (subida) e `onPause` → `onStop` → `onDestroy` (descida); rotação **destrói e recria** a Activity.
- Recursos caros (preview de câmera, renderização) devem pausar em `onStop()` e retomar em `onStart()` — bateria é crítica com os óculos.
- `ViewModel` sobrevive a mudanças de configuração e é o dono do estado da tela e do `viewModelScope`.
- Padrão de estado: `data class UiState` imutável + `MutableStateFlow` privado + `StateFlow` público + `update { it.copy(...) }`.
- UDF: estado desce (`ViewModel` → `UI`), eventos sobem (`UI` → funções do `ViewModel`).
- No Compose, `UI = f(state)`: colete com `collectAsStateWithLifecycle()` e a recomposição atualiza a tela sozinha.

12.8.6 Referências

- ACTIVITY LIFECYCLE / ANDROID DEVELOPERS. Disponível em: <https://developer.android.com/guide/components/activities/activity-lifecycle>. Diagrama oficial e descrição de cada callback do ciclo de vida.
- VIEWMODEL OVERVIEW / ANDROID DEVELOPERS. Disponível em: <https://developer.android.com/topic/libraries/architecture/viewmodel>.

Quando usar ViewModel e como ele sobrevive a mudanças de configuração.

- STATEFLOW AND SHAREDFLOW / ANDROID DEVELOPERS. Disponível em: <https://developer.android.com/kotlin/flow/stateflow-and-sharedflow>. Padrões de exposição de estado com StateFlow, incluindo coleta lifecycle-aware.
- JETPACK COMPOSE / ANDROID DEVELOPERS. Disponível em: <https://developer.android.com/jetpack/compose>. Documentação central do Compose: composables, estado e recomposição.
- APP MANIFEST OVERVIEW / ANDROID DEVELOPERS. Disponível em: <https://developer.android.com/guide/topics/manifest/manifest-intro>. Estrutura e elementos do AndroidManifest.xml.
- CONFIGURE YOUR BUILD / ANDROID DEVELOPERS. Disponível em: <https://developer.android.com/build>. Gradle no Android: módulos, Kotlin DSL e version catalogs.

12.8.6.1 Para ir além



GUIDE TO APP ARCHITECTURE / ANDROID DEVELOPERS. Disponível em: <https://developer.android.com/topic/architecture>. A arquitetura recomendada completa (UI layer, data layer, UDF) — o "mapa" onde este tópico se encaixa.



STATE IN COMPOSE / ANDROID DEVELOPERS. Disponível em: <https://developer.android.com/jetpack/compose/state>. Aprofundamento em estado e recomposição, incluindo state hoisting.

12.9 Permissões em *runtime*

12.9.1 Objetivos de aprendizagem

Ao final, você será capaz de:

- Explicar a diferença entre declarar uma permissão no `AndroidManifest.xml` e solicitá-la em runtime, e por que as duas etapas são obrigatórias para permissões perigosas.
- Implementar o fluxo completo de solicitação de permissões com `ActivityResultContracts`, incluindo checagem prévia e exibição de justificativa (rationale).
- Tratar os três desfechos possíveis de um pedido: concessão, negação simples e negação permanente — direcionando o usuário às configurações do app quando necessário.
- Configurar as permissões exigidas pelo app companion dos óculos: câmera (CAMERA), microfone (RECORD_AUDIO) e Bluetooth (BLUETOOTH_CONNECT/BLUETOOTH_SCAN).

12.9.2 Pré-requisitos

- Tópico 1.1 (Introdução ao Kotlin): lambdas e funções de extensão.
- Tópico 1.3 (Fundamentos de Android): Activity, lifecycle e o papel do `AndroidManifest.xml`.
- Android Studio atualizado e um dispositivo físico ou emulador com Android 12 (API 31) ou superior — recomendado, pois é a partir dessa versão que valem as permissões de Bluetooth que usaremos.

12.9.3 Conceito / fundamentação

12.9.3.1 O modelo em dois tempos: manifesto e runtime

O Android trata permissões em dois momentos distintos. No **manifesto**, o app declara tudo o que *pretende* usar — é como a lista de acessos que uma empresa entrega à portaria do prédio: sem constar na lista, nem adianta pedir. Em **runtime**, para as permissões consideradas sensíveis, o app precisa pedir a autorização ao usuário no momento do uso — é o crachá que a portaria só libera com a sua assinatura, na hora.

Esse modelo existe desde o Android 6.0 (API 23). Antes disso, todas as permissões eram concedidas na instalação; hoje, as sensíveis exigem consentimento explícito, e o usuário pode revogá-las a qualquer momento nas

configurações. Consequência direta para o seu código: **nunca assuma que uma permissão concedida ontem continua concedida hoje** — cheque sempre antes de usar o recurso.

12.9.3.2 Permissões normais x perigosas

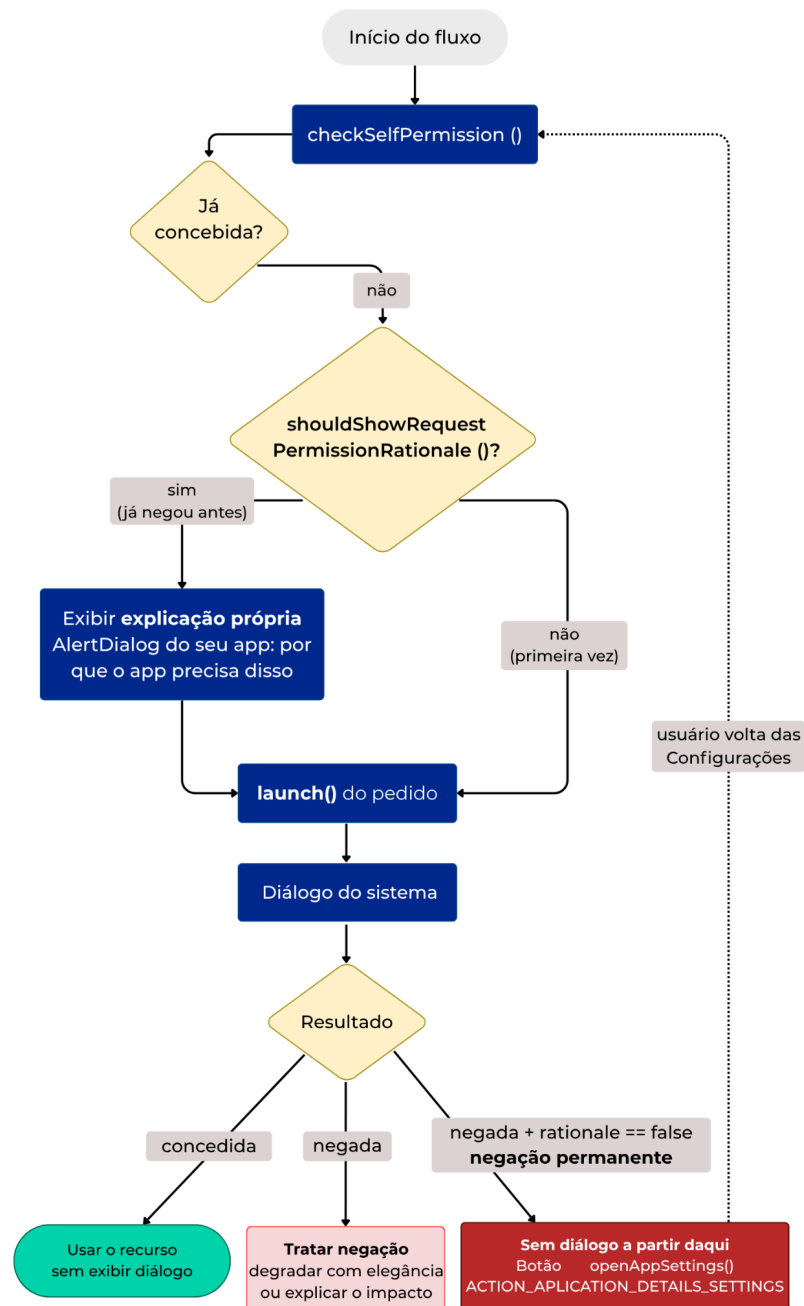
- **Normais** (ex.: `INTERNET`): risco baixo. Basta declarar no manifesto; o sistema concede automaticamente na instalação.
- **Perigosas** (ex.: `CAMERA`, `RECORD_AUDIO`): dão acesso a dados ou hardware sensíveis. Exigem declaração no manifesto e pedido em runtime.
- Permissões perigosas são organizadas em grupos (câmera, microfone, "dispositivos por perto"...). O diálogo do sistema é exibido por permissão solicitada, mas o agrupamento influencia como o usuário vê o pedido na tela de configurações.

12.9.3.3 O fluxo de solicitação

O fluxo correto tem quatro decisões encadeadas (Figura 7):

1. **Checar**: a permissão já foi concedida? Se sim, use o recurso direto — não exiba diálogo à toa.
2. **Justificar (rationale)**: se o usuário já negou uma vez, o sistema sinaliza que vale explicar *por que* o app precisa daquilo antes de pedir de novo.
3. **Pedir**: dispare o diálogo do sistema.
4. **Tratar o resultado**: concedida → prossiga; negada → degrade a experiência com elegância ou explique o impacto; negada permanentemente → oriente o usuário a habilitar manualmente nas configurações.

Figura 7 – Fluxo de solicitação de permissão em runtime



Fonte: autoria própria.

12.9.3.4 Negação simples, negação permanente e "apenas desta vez"

Três desfechos negativos ou parciais merecem atenção:

- **Negação simples:** o usuário tocou "Não permitir" uma vez. Você ainda pode pedir de novo (idealmente após explicar o motivo).

- **Negação permanente:** a partir do Android 11 (API 30), se o usuário nega a mesma permissão repetidamente, o sistema passa a **não exibir mais o diálogo** — o pedido retorna negado na hora, silenciosamente. Em versões anteriores, isso acontecia via checkbox "Não perguntar novamente". Não existe API para "desnegar": o único caminho é o usuário habilitar manualmente em Configurações > Apps.
- **"Apenas desta vez":** no Android 11+, para câmera, microfone e localização o usuário pode conceder acesso temporário. A permissão é revogada automaticamente depois que o app sai de primeiro plano por um tempo — mais um motivo para checar antes de cada uso, e nunca cachear o resultado.

Nota11: o Android 11+ também revoga automaticamente permissões de apps que ficam meses sem uso (auto-reset). Para um app de hackathon isso raramente aparece, mas explica por que apps "perdem" permissões com o tempo.

12.9.3.5 As permissões do programa dos óculos

O app companion conversa com os óculos via Bluetooth e processa visão e voz on-device — como visto no 1.3, ele é um app Android comum, então está sujeito a exatamente este modelo de permissões. As que nos interessam estão descritas na Tabela 5.

Tabela 5 – Permissões Android utilizadas pelo aplicativo companion

Permissão	Para que serve no programa	Tipo
CAMERA	Acesso à câmera (no telefone, a câmera local; o acesso ao stream da câmera dos óculos via SDK será visto no tópico 2.5)	Perigosa — runtime
RECORD_AUDIO	Captura de áudio, incluindo áudio que chega por Bluetooth a partir do array de microfones dos óculos (detalhes no tópico 2.6)	Perigosa — runtime
BLUETOOTH_CONNECT	Conectar-se a dispositivos pareados — é ela que permite	Perigosa — runtime, grupo "Dispositivos por perto"

	falar com os óculos (Android 12+)	
BLUETOOTH_SCAN	Procurar dispositivos Bluetooth próximos (Android 12+)	Perigosa — runtime, grupo "Dispositivos por perto"
BLUETOOTH / BLUETOOTH_ADMIN	Equivalentes legadas para Android 11 ou inferior	Normais — só manifesto (com <code>maxSdkVersion="30"</code>)

Fonte: autoria própria.

Antes do Android 12, escanear Bluetooth exigia também permissão de localização (`ACCESS_FINE_LOCATION`), porque beacons permitem inferir posição. No Android 12+, declarar `BLUETOOTH_SCAN` com a flag `neverForLocation` elimina essa exigência.

O app companion vive sob **dois** modelos de permissão que funcionam em paralelo:

1. **Permissões do Android** (o assunto deste tópico). O DAT exige apenas `BLUETOOTH`, `BLUETOOTH_CONNECT` e `INTERNET`. Note o que **não** está na lista: consumir a câmera e os microfones dos óculos **não** exige `CAMERA` nem `RECORD_AUDIO` no telefone, porque quem fala com o hardware é o app Meta AI — o seu app recebe os dados já entregues pelo toolkit. `CAMERA` só entra se você usar a câmera do próprio celular como feed do Mock Device (tópico 2.7); `RECORD_AUDIO` só entra no caminho de áudio por Bluetooth com `AudioRecord` (tópico 2.6).
2. **Permissões do toolkit**, concedidas dentro do app Meta AI e não pelo diálogo do Android. Elas exigem que seu app esteja **registrado** primeiro — sem registro, o pedido falha; com registro mas sem permissão, o app conecta e não acessa a câmera. E o usuário escolhe entre **permitir uma vez** (temporário) ou **permitir sempre** (persistente): o mesmo dilema do "Apenas desta vez" que você viu acima, com a mesma consequência prática — **cheque antes de usar, nunca cacheie**. O fluxo completo é o tópico 2.4.

A lição deste tópico vale para as duas camadas: o mecanismo é sempre *cheçar* → *justificar* → *pedir* → *tratar o desfecho*. O que muda é quem exhibe o diálogo.

Na prática: no dia do hackathon, a primeira coisa que seu app faz é conectar-se aos óculos. Se `BLUETOOTH_CONNECT` não tiver sido concedida, a conexão falha com `SecurityException` antes de qualquer modelo de IA rodar. Trate as permissões como o "portão de entrada" da sua demo: peça todas no onboarding, valide antes de cada fluxo e tenha uma tela de recuperação para o caso de negação.

12.9.4 Na prática

Vamos montar o fluxo completo em uma Activity: declarar, checar, justificar, pedir e tratar todos os desfechos.

12.9.4.1 Passo 1 — Declarar no manifesto

► **Código 1.4-01** – AndroidManifest.xml, dentro de `<manifest>` — código completo no notebook companion (seção 1.4.4.1).

Atenção! Esquecer a declaração no manifesto é o erro mais silencioso deste capítulo. O pedido em runtime **não quebra** — o diálogo simplesmente nunca aparece e o resultado volta negado na hora. Se o seu pop-up "não abre", confira o manifesto antes de qualquer outra coisa.

12.9.4.2 Passo 2 — Função de checagem

► **Código 1.4-02** – código completo no notebook companion (seção 1.4.4.2).

12.9.4.3 Passo 3 — Montar a lista de permissões do programa

As permissões de Bluetooth só existem a partir da API 31, então a lista depende da versão do sistema:

► **Código 1.4-03** – código completo no notebook companion (seção 1.4.4.3).

12.9.4.4 Passo 4 — Registrar o launcher e tratar o resultado

A API moderna é `registerForActivityResult` com o contrato `RequestMultiplePermissions`. O registro **precisa** acontecer na inicialização da Activity (como propriedade), antes de ela chegar ao estado STARTED:

► **Código 1.4-04** – código completo no notebook companion (seção 1.4.4.4).

XXX:Atenção! Dentro do callback de resultado, `shouldShowRequestPermissionRationale(permissoes)` retornando `false` após uma negação é o sinal de negação permanente. Não existe método `isPermanentlyDenied()` no framework — essa combinação (negado + rationale falso) é a forma padrão de detectar.

12.9.4.5 Passo 5 — Checar, justificar e pedir

► **Código 1.4-05** – código completo no notebook companion (seção 1.4.4.5).

O diálogo de rationale é um diálogo seu (um `AlertDialog` simples resolve), explicando em uma frase por que o app precisa do acesso — por exemplo: "Precisamos do Bluetooth para conectar aos seus óculos e do microfone para ouvir seus comandos de voz".

12.9.4.6 Passo 6 — Tratar a negação permanente

Quando a negação é permanente, chamar `launch()` de novo é inútil: o resultado volta negado sem diálogo. O único caminho é levar o usuário à tela de detalhes do app:

► **Código 1.4-06** – código completo no notebook companion (seção 1.4.4.6).

Na UI, mostre uma mensagem clara ("Permissão de Bluetooth desativada. Habilite em Configurações para conectar aos óculos") com um botão que chama `openAppSettings()`. Ao voltar da tela de configurações, cheque as permissões de novo (por exemplo, no `onResume()` ou ao reentrar no fluxo) — o usuário pode ou não ter habilitado.

Atenção! Negação permanente é o caso que mais derruba demo de hackathon! Durante os testes, é comum negar o diálogo algumas vezes "só para ver" — e aí o pop-up para de aparecer para sempre naquele aparelho, sem nenhum aviso. Se as permissões "sumiram", vá em Configurações > Apps > seu app > Permissões e habilite manualmente, ou reinstale o app para zerar o estado. No código, sempre implemente o caminho de recuperação via `openAppSettings()`.

12.9.4.7 Erros comuns

- **Diálogo nunca aparece, resultado sempre negado** → permissão fora do manifesto (ou negação permanente já ativada). Confira o manifesto primeiro; depois, o estado em Configurações.
- `IllegalStateException` **ao registrar o launcher** → `registerForActivityResult` foi chamado tarde demais (ex.: dentro de um click listener). Registre sempre como propriedade da Activity/Fragment.
- `SecurityException` **ao conectar no Android 12+** → `BLUETOOTH_CONNECT` declarada mas não concedida em runtime, ou ausente do manifesto. As permissões legadas `BLUETOOTH/BLUETOOTH_ADMIN` não valem na API 31+.
- **Permissão "some" entre sessões** → o usuário concedeu "Apenas desta vez". Nunca cacheie o resultado; cheque com `hasPermission()` a cada uso.

Nota 12: em Jetpack Compose, o equivalente é `rememberLauncherForActivityResult(ActivityResultContracts.RequestMultiplePermissions()) { ... }` — a lógica de checagem, rationale e tratamento é idêntica.

Na prática: estruture o app companion para pedir as permissões em uma tela de onboarding dedicada, antes da primeira tentativa de conexão com os óculos — e não espalhadas pelo app. Assim, quando o fluxo de registro e sessão do SDK entrar em cena (tópico 2.4), o terreno já está limpo, e a banca do hackathon nunca verá um pop-up de permissão no meio da demo.

12.9.5 Resumo / cheatsheet

- Permissão perigosa = declarar no manifesto e pedir em runtime (desde o Android 6.0/API 23); só manifesto não basta.

- Fluxo: `checkSelfPermission` → `rationale` (se já negou) → `launch()` → tratar resultado.
- API moderna:
`registerForActivityResult(ActivityResultContracts.RequestMultiplePermissions())`, registrado como propriedade da Activity.
- Permissões do programa: `CAMERA`, `RECORD_AUDIO`, `BLUETOOTH_CONNECT` e `BLUETOOTH_SCAN` (Android 12+; legadas `BLUETOOTH/BLUETOOTH_ADMIN` com `maxSdkVersion="30"`).
- `BLUETOOTH_SCAN` com `neverForLocation` dispensa permissão de localização no Android 12+.
- Negação permanente: negado + `shouldShowRequestPermissionRationale == false` → diálogo não abre mais; recuperação só via Configurações (`ACTION_APPLICATION_DETAILS_SETTINGS`).
- "Apenas desta vez" (Android 11+) revoga câmera/microfone automaticamente — cheque a permissão antes de **cada** uso, nunca cacheie.
- Permissão fora do manifesto = pedido negado silenciosamente, sem diálogo.

12.9.6 Referências

- PERMISSIONS ON ANDROID / ANDROID DEVELOPERS. Disponível em: <https://developer.android.com/guide/topics/permissions/overview> — visão geral do modelo de permissões: tipos, grupos e ciclo de vida.
- REQUEST RUNTIME PERMISSIONS / ANDROID DEVELOPERS. Disponível em: <https://developer.android.com/training/permissions/requesting> — fluxo oficial de solicitação com `ActivityResultContracts`, `rationale` e tratamento de negação.
- BLUETOOTH PERMISSIONS / ANDROID DEVELOPERS. Disponível em: <https://developer.android.com/develop/connectivity/bluetooth/bt-permissions>

— BLUETOOTH_CONNECT/BLUETOOTH_SCAN, flag neverForLocation e compatibilidade com versões antigas.

12.9.6.1 Para ir além



APP PERMISSIONS BEST PRACTICES / ANDROID DEVELOPERS.

Disponível em:

<https://developer.android.com/training/permissions/usage-notes> — boas práticas de UX: quando pedir, como explicar e como degradar sem a permissão.



META WEARABLES DEVELOPER CENTER. Disponível em:

<https://developers.meta.com/wearables> — documentação oficial do Device Access Toolkit, incluindo requisitos do app companion.

12.10 Background e eficiência de energia

12.10.1 Objetivos de aprendizagem

Ao final, você será capaz de:

- Explicar as restrições do Android para execução em segundo plano e por que elas existem.
- Comparar foreground services e WorkManager e escolher o mecanismo certo para cada tarefa do app companion.
- Implementar um foreground service com o tipo correto e uma tarefa adiável com WorkManager e constraints.
- Explicar o throttling térmico e por que a inferência "em rajada" (bursty) supera a inferência contínua em bateria e desempenho.
- Aplicar boas práticas de economia de energia em pipelines de IA on-device e medir o consumo do seu app.

12.10.2 Pré-requisitos

- Tópico 1.2 — Coroutines e Flow (usaremos `CoroutineWorker`, `delay` e operadores de Flow).

- Tópico 1.3 — Fundamentos de Android (components, lifecycle, notificações).
- Tópico 1.4 — Permissões em runtime (`POST_NOTIFICATIONS` e permissões de microfone já vistas lá).
- Android Studio atualizado, com um projeto usando `targetSdk` 34 ou superior.

12.10.3 Conceito / fundamentação

12.10.3.1 O problema: seu app não está sempre na tela

No hackathon, o app companion é o cérebro da solução: os óculos capturam imagem e áudio, enviam via Bluetooth, e é o celular que roda os modelos de IA. Só que o usuário não vai ficar com o app aberto na tela enquanto caminha pela rua — o celular estará no bolso, com a tela apagada. Se o seu pipeline morrer quando o app sai de primeiro plano, a solução inteira para de funcionar.

Ao mesmo tempo, o Android limita agressivamente o que apps podem fazer em segundo plano, justamente para proteger a bateria. Esse é o equilíbrio deste tópico: manter o pipeline vivo e gastar o mínimo de energia possível.

12.10.3.2 O que o Android faz com apps em segundo plano

Desde o Android 8, o sistema impõe limites severos: processos em background podem ser encerrados a qualquer momento, e serviços comuns iniciados em background são proibidos. Além disso, quando o aparelho fica parado com a tela apagada, entra em **Doze mode**: o sistema adia acesso à rede, jobs e alarmes para janelas curtas de manutenção. O **App Standby** completa o cerco, rebaixando apps pouco usados para "buckets" com menos direitos de execução.

A consequência prática: você não escolhe "rodar em background" livremente. Você escolhe entre dois mecanismos sancionados pelo sistema — foreground service ou WorkManager — de acordo com a natureza da tarefa.

12.10.3.3 Foreground service: trabalho contínuo e visível

Um **foreground service** é um serviço que roda com uma notificação permanente visível ao usuário. Em troca dessa transparência, o sistema o trata

como quase tão importante quanto o app em primeiro plano: ele não é adiado pelo Doze e raramente é morto.

Use foreground service quando a tarefa é **contínua, iniciada pelo usuário e perceptível por ele**: reproduzir áudio, rastrear localização em um treino ou — no nosso caso — manter a sessão com os óculos ativa, recebendo o streaming de áudio via Bluetooth e processando-o em tempo real.

Desde o Android 14 (API 34), todo foreground service precisa declarar um **tipo** (`foregroundServiceType`) que descreve o que ele faz — `microphone`, `camera`, `connectedDevice`, `dataSync` etc. — e a permissão correspondente no manifest. O sistema usa o tipo para aplicar regras específicas: por exemplo, um serviço do tipo `microphone` só tem acesso ao microfone se for iniciado enquanto o app está em primeiro plano.

12.10.3.4 WorkManager: trabalho adiável e garantido

O **WorkManager** é a biblioteca do Jetpack para trabalho **adiável e garantido**: tarefas que não precisam rodar agora, mas precisam rodar em algum momento — mesmo que o app seja fechado ou o aparelho reinicie. Você descreve a tarefa em um `Worker`, define **constraints** (só com carregador, só no Wi-Fi, só com bateria não baixa) e o sistema escolhe o melhor momento de execução, respeitando Doze e App Standby.

No contexto do programa, é o mecanismo ideal para: baixar ou atualizar arquivos de modelo, sincronizar resultados com um servidor, limpar *caches* de *frames* e gerar relatórios de uso.

12.10.3.5 Como escolher

Na Tabela 6 estão os critérios que orientam a decisão por *foreground service* ou *WorkManager*.

Tabela 6 – Comparação entre Foreground Service e WorkManager

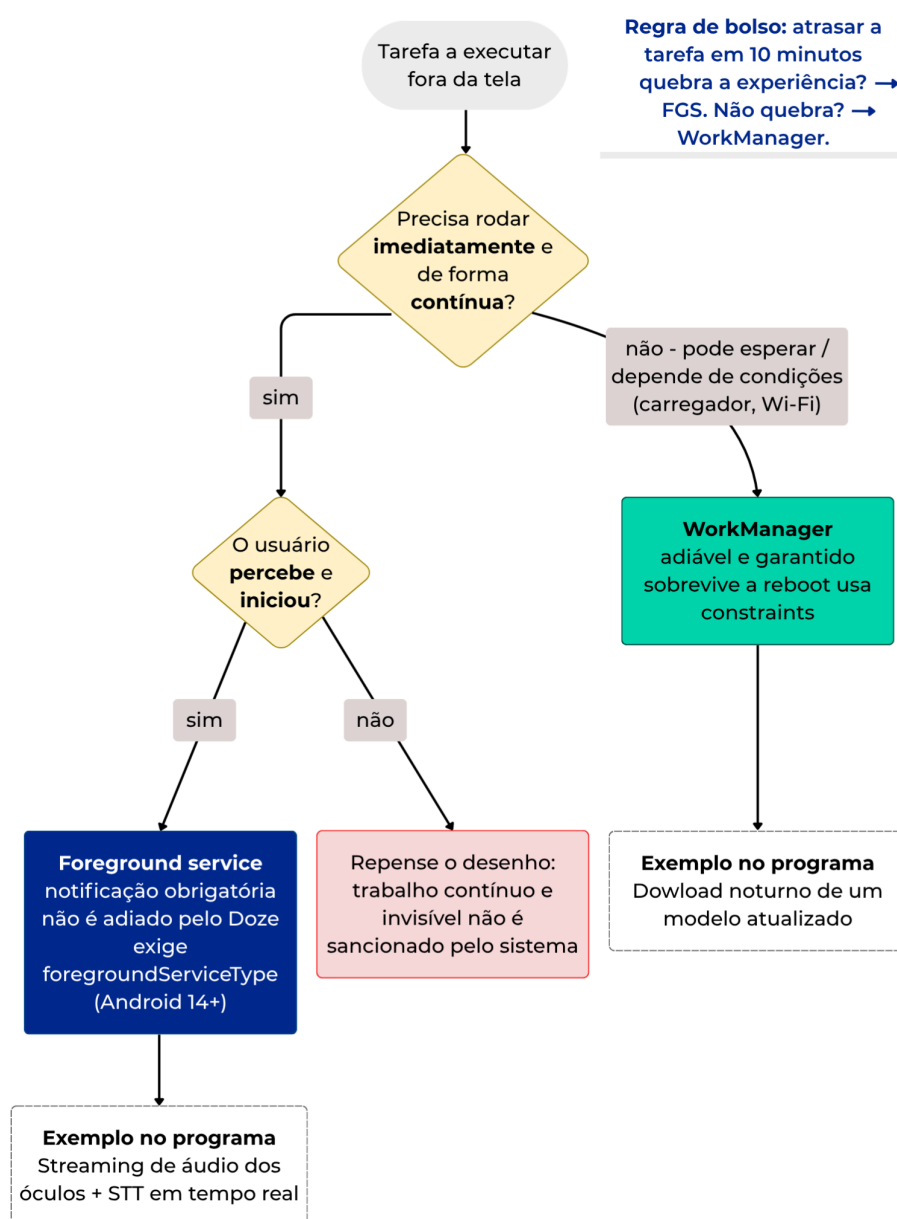
Critério	Foreground service	WorkManager
Quando roda	Agora, continuamente	Quando o sistema permitir
Usuário percebe?	Sim (notificação obrigatória)	Não necessariamente

Sobrevive a <i>reboot</i> ?	Não (precisa ser reiniciado)	Sim (trabalho persistido)
Adiável?	Não	Sim (<i>constraints</i>)
Exemplo no programa	<i>Streaming</i> de áudio dos óculos + STT em tempo real	<i>Download</i> noturno de um modelo atualizado

Fonte: autoria própria.

Regra de bolso: se atrasar a tarefa em 10 minutos quebra a experiência, é foreground service; se não quebra, é WorkManager (Figura 8).

Figura 8 – Foreground service ou WorkManager?



Fonte: autoria própria.

12.10.3.6 Por que inferência "em rajada" (bursty)

Rodar um modelo de IA é a operação mais cara em energia que o app companion faz. A intuição ingênua — "quanto mais frames por segundo eu processar, melhor o produto" — é uma armadilha dupla:

1. **Bateria:** cada inferência consome uma quantidade quase fixa de energia. Inferência contínua a 30 fps consome $\sim 30\times$ mais do que 1 inferência por segundo, quase sempre sem ganho real de experiência.
2. **Calor:** o SoC do celular não tem ventoinha; ele dissipa calor passivamente pela carcaça. Sob carga contínua, a temperatura sobe até o sistema se proteger.

A estratégia **bursty** inverte a lógica: rode o modelo em picos curtos e intensos — quando um gatilho acontece (o usuário fala um comando, uma foto chega dos óculos) — e deixe o hardware ocioso e esfriando no resto do tempo. Isso segue o princípio de *race to sleep*: terminar o trabalho o mais rápido possível para o chip voltar logo ao estado de baixo consumo, em vez de trabalhar devagar por muito tempo.

Analogia: um corredor consegue dar vários sprints de 10 segundos ao longo de uma hora, com pausas para recuperar. Se tentar correr a velocidade de sprint por uma hora contínua, o corpo entra em colapso e ele termina andando — mais devagar do que se tivesse alternado.

12.10.3.7 Throttling térmico, explicado de forma simples

Quando a temperatura do SoC passa de certos limiares, o sistema reduz a frequência (clock) da CPU/GPU/NPU para gerar menos calor. Isso é o **throttling térmico**. O efeito em cascata é perverso para IA on-device:

- O clock cai → cada inferência demora mais.
- Inferências mais lentas mantêm o chip ocupado por mais tempo → mais energia gasta por inferência.
- O chip ocupado por mais tempo continua gerando calor → o throttling se aprofunda.

Ou seja: a inferência contínua não só drena a bateria — ela deixa o próprio modelo mais lento com o passar dos minutos. Uma demo que roda a 15 fps no primeiro minuto pode estar a 5 fps no décimo, com o celular quente na mão do jurado.

O Android expõe o estado térmico ao app: `PowerManager.getThermalHeadroom()` retorna uma previsão de quão perto do limite de throttling severo o aparelho estará (1.0 = no limite), e `addThermalStatusListener` notifica mudanças de status térmico. Você verá o uso na seção prática.

Na prática: este tópico é o coração do checkpoint de "**Eficiência de bateria**" do camp. A avaliação vai olhar exatamente para as decisões ensinadas aqui: seu pipeline usa foreground service só para o que é contínuo? O trabalho adiável tem constraints? A inferência dispara por gatilho ou roda em loop cego? Um pipeline contínuo de visão pode derrubar a bateria do celular em poucas horas — e lembre-se de que a bateria dos óculos também é finita e o streaming Bluetooth constante a consome; capturar sob demanda poupa os dois lados do link.

12.10.3.8 Boas práticas de economia para IA on-device

- **Gatilho, não loop:** dispare inferência por evento (comando de voz, toque, chegada de foto), nunca em `while(true)`.
- **Reduza a entrada:** menos resolução e menos frames por segundo = menos energia por resultado. Comece pequeno e só aumente se a qualidade exigir.
- **Limite a taxa:** imponha um teto de inferências por segundo (você implementará com `sample` de Flow, visto no 1.2).
- **Respeite o estado térmico:** consulte o thermal headroom antes de bursts pesados e recue quando o aparelho estiver quente.
- **Use modelos otimizados:** modelos quantizados e delegates de hardware (NPU/GPU) reduzem tempo e energia por inferência — os detalhes foram vistos mais a frente.
- **Libere recursos:** feche interpretadores, streams de áudio e conexões quando o burst termina.

- **Meça, não adivinhe:** use o Power Profiler do Android Studio e o Battery Historian para ver onde a energia realmente vai.

12.10.4 Na prática

12.10.4.1 Passo 1 — Foreground service para o pipeline de áudio

Cenário: o usuário ativou o modo "assistente" e o app precisa continuar recebendo áudio dos óculos via Bluetooth e transcrevendo, mesmo com a tela apagada.

Manifest — permissões e declaração do serviço com tipos:

► **Código 1.5-01** – Permissão geral de FGS + permissões por tipo (Android 14+) — código completo no notebook companion (seção 1.5.4.1).

O serviço:

► **Código 1.5-02** – código completo no notebook companion (seção 1.5.4.1).

E o start, feito **enquanto o app está visível** (por exemplo, no clique do botão "ativar assistente"):

```
// Em uma Activity visível:  
startForegroundService(Intent(this, AudioPipelineService.class.java))
```

Código 1.5-03 · também no notebook companion, seção 1.5.4.1

► **Nota13:** o comportamento dos tipos de FGS depende da versão do Android do aparelho de teste (validado no Android 16).

► **Atenção!** Dois erros clássicos aqui:

1) Tentar iniciar o foreground service com o app em background lança `ForegroundServiceStartNotAllowedException` no Android 12+ — sempre inicie a partir de uma tela visível ou de um gatilho permitido pelo sistema.

2) Esquecer o `foregroundServiceType` no manifest ou na chamada de `startForeground` gera exceção no Android 14+ — declare nos dois lugares.

12.10.4.2 Passo 2 — WorkManager para o trabalho adiável

Cenário: baixar uma versão atualizada do modelo, mas só com o celular carregando e no Wi-Fi — nunca no meio da demo.

Dependência (`build.gradle.kts` do módulo):

```
implementation("androidx.work:work-runtime-ktx:2.11.2")
```

Código 1.5-04 · também no notebook companion, seção 1.5.4.2

O worker e o agendamento:

► **Código 1.5-05** – código completo no notebook companion (seção 1.5.4.2).

Atenção! Não use `WorkManager` para nada que o usuário está esperando em tempo real. Com Doze e App Standby, "adiável" pode significar horas de espera — é o comportamento esperado, não um bug.

12.10.4.3 Passo 3 — Inferência bursty com limite de taxa e freio térmico

Cenário: frames chegam dos óculos e você roda um modelo de visão, mas com teto de taxa e recuo quando o aparelho esquenta.

► **Código 1.5-06** – código completo no notebook companion (seção 1.5.4.3).

Nota 14: `getThermalHeadroom` exige API 30+ e pode retornar NaN em aparelhos sem suporte; há limite de frequência de chamadas. Trate NaN como "sem informação" e mantenha só o limite de taxa (Validado no Android 16).

Na prática: para o checkpoint de bateria, este padrão é o seu argumento técnico: mostre que o app processa sob demanda, com teto de taxa e freio térmico. Uma demo que continua rápida e com o celular frio depois de 10 minutos vale mais do que qualquer slide.

12.10.4.4 Passo 4 — Medir o consumo

Antes de otimizar, meça:

1. **Power Profiler** (Android Studio): mostra consumo estimado por recurso (CPU, rede, GPU) em tempo real enquanto você exercita o app.

2. **Battery Historian**: colete com `adb shell dumpsys batterystats` (após `adb shell dumpsys batterystats --reset` e uma sessão de uso) e visualize a linha do tempo de wakeups, jobs e serviços do seu app.

Compare uma sessão com inferência contínua e outra com o padrão bursty — a diferença aparece na primeira medição.

12.10.5 Resumo / cheatsheet

- Android restringe background (Doze, App Standby); os caminhos sancionados são foreground service e WorkManager.
- Foreground service = contínuo, imediato, com notificação; exige `foregroundServiceType` + permissão por tipo no Android 14+.
- WorkManager = adiável e garantido; use constraints (carregador, Wi-Fi, bateria) e trabalho único nomeado.
- Regra de bolso: atrasar 10 min quebra a experiência? FGS. Não quebra? WorkManager.
- Bursty > contínuo: picos curtos + ociosidade = menos energia e sem throttling (*race to sleep*).
- Throttling térmico: calor → clock menor → inferência mais lenta → mais calor; contínuo entra nessa espiral.
- Freios no pipeline: gatilho em vez de loop, `sample` para teto de taxa, `getThermalHeadroom` para recuo.
- Meça com Power Profiler e Battery Historian antes e depois de otimizar.

12.10.6 Referências

- FOREGROUND SERVICES / ANDROID DEVELOPERS. Disponível em: <https://developer.android.com/develop/background-work/services/foreground-services>. Tipos de serviço, permissões por tipo e restrições de start por versão.

- WORKMANAGER / ANDROID DEVELOPERS. Disponível em:
<https://developer.android.com/topic/libraries/architecture/workmanager>.
Guia da biblioteca: workers, constraints, trabalho único e periódico.
- BACKGROUND WORK OVERVIEW / ANDROID DEVELOPERS.
Disponível em:
<https://developer.android.com/develop/background-work/background-tasks>.
Árvore de decisão oficial entre os mecanismos de execução em segundo plano.
- OPTIMIZE FOR DOZE AND APP STANDBY / ANDROID DEVELOPERS.
Disponível em:
<https://developer.android.com/training/monitoring-device-state/doze-standby>. Como Doze e App Standby afetam seu app e como testá-los com adb.
- POWERMANAGER / ANDROID DEVELOPERS. Disponível em:
<https://developer.android.com/reference/android/os/PowerManager>.
Referência de getThermalHeadroom e addThermalStatusListener.
- THERMAL API / ANDROID DEVELOPERS. Disponível em:
<https://developer.android.com/games/optimize/adpf/thermal>. Guia de adaptação de carga ao estado térmico (escrito para jogos, vale para inferência).

12.10.6.1 Para ir além



BATTERY HISTORIAN / GOOGLE (GITHUB). Disponível em:
<https://github.com/google/battery-historian>. Ferramenta de visualização dos batterystats coletados via adb.



APP STANDBY BUCKETS / ANDROID DEVELOPERS.
Disponível em:
<https://developer.android.com/topic/performance/appstandby>.
Como o sistema classifica apps por uso e o que cada bucket permite.



LITERT / GOOGLE AI EDGE. Disponível em:
<https://ai.google.dev/edge/litert>. Runtime de inferência on-device e suas opções de aceleração de hardware.

12.11 Edge AI: otimização e redução de modelos

12.11.1 Objetivos de aprendizagem

Ao final, você será capaz de:

- **Explicar** o que é Edge AI e ponderar seus benefícios (latência, privacidade, offline, custo) contra seus limites (memória, calor, energia).
- **Comparar** as técnicas de redução de modelos — quantização (post-training × quantization-aware), destilação, pruning e LoRA/adaptadores — e identificar quando cada uma compensa.
- **Diferenciar** os formatos `.tflite`/`.litemlm`, GGUF e ONNX, associando cada um ao runtime e ao hardware onde roda melhor.
- **Distinguir** CPU, GPU, NPU e TPU e escolher o alvo de execução adequado para celular × notebook.
- **Montar** a combinação otimização + formato + hardware que encaixa um modelo no hardware disponível no hackathon, priorizando latência.

12.11.2 Pré-requisitos

- **Tópico 1.3 (Fundamentos de Android)** — noção de app, processo e recursos compartilhados do aparelho.
- **Tópico 1.5 (Background e bateria)** — entender por que trabalho computacional contínuo custa energia e gera calor.
- **Ferramentas:** nenhuma é obrigatória para acompanhar este tópico (ele é conceitual). Para reproduzir os exemplos da seção 5: Android Studio (versão estável mais recente) e um celular físico Android.

12.11.3 Conceito / fundamentação

12.11.3.1 O que é Edge AI

Edge AI é rodar modelos de IA **no próprio dispositivo** — celular, notebook, óculos — em vez de enviar os dados para um servidor na nuvem e esperar a resposta. "Edge" (borda) é o nome dado a esses dispositivos na ponta da rede, perto de quem gera os dados.

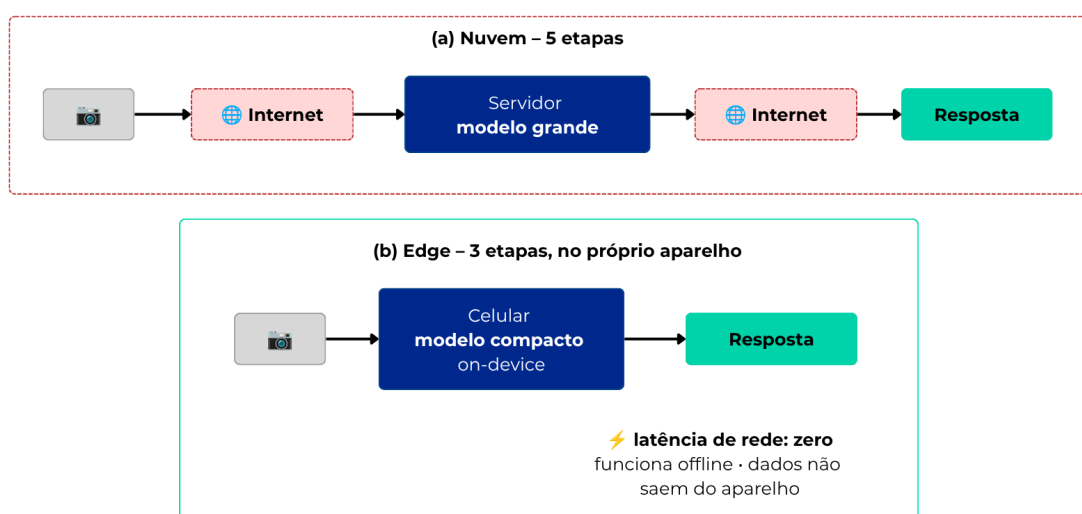
Uma analogia: nuvem é pedir comida por delivery — a cozinha é enorme e faz qualquer prato, mas você paga o frete (rede) e espera a entrega (latência). Edge AI é cozinhar em casa — a cozinha é menor e o cardápio mais limitado, mas o prato sai na hora, ninguém vê o que você come e funciona mesmo sem "entregador" disponível.

Os benefícios, um a um:

- **Latência:** sem viagem de ida e volta pela internet; a resposta depende só do hardware local. Para experiências em tempo real, isso é decisivo.
- **Privacidade:** os dados (imagens, áudio) não saem do aparelho. Importante quando a câmera está apontada para o mundo ao redor do usuário.
- **Offline:** funciona sem conexão — em um evento lotado com Wi-Fi saturado, isso deixa de ser detalhe.
- **Custo:** sem servidor para pagar e sem cobrança por chamada de API. E os limites, que são o motivo de este tópico existir:
- **Memória:** a RAM do celular é compartilhada com o sistema e os outros apps; um modelo de bilhões de parâmetros em precisão cheia simplesmente não cabe.
- **Calor:** inferência contínua esquentando o SoC; o sistema reage com *thermal throttling* (reduz a frequência dos chips), e a latência piora justamente quando o app mais roda.
- **Energia:** cada inferência consome bateria — e, como visto no 1.5, o Android pune apps que drenam energia em segundo plano.

Na Figura 9, veja uma comparação do custo da viagem pela rede em nuvem e Edge AI.

Figura 9 – Nuvem × Edge AI: o custo da viagem pela rede



Fonte: autoria própria.

Na prática: na arquitetura do programa, os óculos Ray-Ban Meta não executam modelos — eles capturam (câmera de 12 MP, 5 microfones) e reproduzem (alto-falantes open-ear), conversando por Bluetooth com o app companion Android. **Toda a IA roda no celular.** O usuário fala ou aponta o olhar, e espera a resposta em áudio: cada milissegundo de inferência entra na conta da experiência. Edge AI não é uma opção estética no hackathon — é a arquitetura obrigatória, e otimizar modelos é o que a torna viável.

12.11.3.2 Por que modelos precisam "emagrecer": a conta da memória

Um modelo de rede neural é, na essência, um conjunto enorme de números chamados **pesos** (*weights*). Por padrão, cada peso é armazenado como um número de ponto flutuante de 32 *bits* — **FP32**, 4 *bytes*. A conta do tamanho mínimo do modelo é direta:

Memória dos pesos $\approx$ número de parâmetros $\times$ bytes por parâmetro

Na Tabela 7, a seguir, você encontra exemplos comparativos de diferentes tamanhos de modelos.

Tabela 7 – Exemplos comparativos de diferentes tamanhos de modelos

Modelo (parâmetros)	FP32 (4 B)	FP16 (2 B)	INT8 (1 B)	INT4 (0,5 B)
---------------------	------------	------------	------------	--------------

1 bilhão (1B)	~4 GB	~2 GB	~1 GB	~0,5 GB
3 bilhões (3B)	~12 GB	~6 GB	~3 GB	~1,5 GB
7 bilhões (7B)	~28 GB	~14 GB	~7 GB	~3,5 GB

Nota: Valores aproximados, apenas para os pesos; a execução ainda exige memória extra para ativações e caches. Fonte: autoria própria.

Repare: um modelo 3B em FP32 exige ~12 GB só de pesos — mais RAM do que muitos celulares têm no total. O mesmo modelo em INT4 cai para ~1,5 GB e passa a ser plausível.

E não é só questão de "caber": em dispositivos móveis, a **largura de banda de memória** (velocidade com que os pesos são lidos da RAM) costuma ser o gargalo da inferência. Pesos menores = menos bytes trafegando = inferência mais rápida. Reduzir tamanho e ganhar latência andam juntos.

12.11.3.3 Quantização: reduzir a precisão dos pesos

Quantização é representar os pesos (e, em alguns esquemas, também as ativações) com menos bits: FP32 → INT8 → INT4. A analogia clássica é a foto: o arquivo RAW da câmera tem informação de sobra; um JPEG bem comprimido é várias vezes menor e, para quase todos os usos, ninguém nota diferença. A quantização faz o mesmo com os números do modelo — sacrifica precisão numérica que o modelo, na maioria dos casos, nem estava usando de fato.

Há duas famílias de abordagem:

- **Post-training quantization (PTQ)** — quantiza um modelo **já treinado**. É o caminho barato: não exige retreinar, roda em minutos com ferramentas prontas e, para INT8, a perda de qualidade costuma ser pequena. Quanto mais agressiva a redução (INT4 ou menos), maior o risco de degradação perceptível.
- **Quantization-aware training (QAT)** — **simula a quantização durante o treinamento** (ou em um fine-tuning), para que o modelo aprenda a compensar o erro de arredondamento. Recupera qualidade em precisões baixas, mas exige pipeline de treino, dados e tempo de computação.

O *trade-off* central — **precisão × tamanho × velocidade** (Tabela 8):

Tabela 8 – Comparação entre níveis de precisão quanto ao tamanho, à velocidade, à qualidade e ao custo de obtenção

Precisão	Tamanho relativo	Velocidade	Qualidade	Custo de obter
FP32	1× (referência)	mais lento	máxima	nenhum (original)
FP16	~0,5×	melhor em GPU	quase idêntica	conversão trivial
INT8 (PTQ)	~0,25×	ganho típico de 2–3× em CPU	perda geralmente pequena	baixo (sem retreino)
INT4	~0,125×	mais rápido ainda em runtimes com suporte	perda pode ser visível	baixo (PTQ) ou médio (QAT para recuperar)

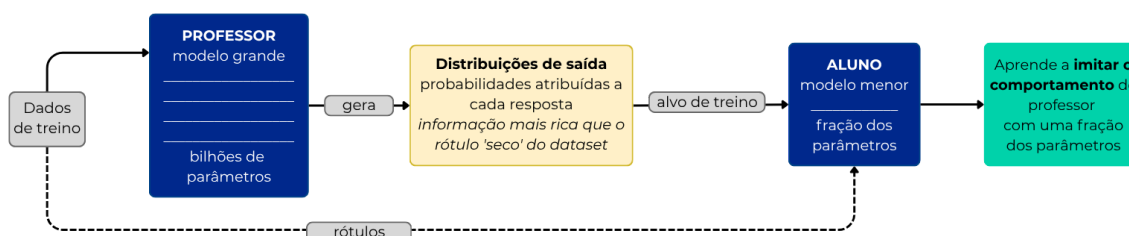
Fonte: autoria própria.

Atenção! Quantização não é grátis. Modelos pequenos e tarefas sensíveis (números, nomes próprios, detalhes finos de imagem) tendem a sofrer mais em INT4. Nunca assuma que "continua bom": meça a qualidade **depois** de quantizar, na sua tarefa, antes de adotar a versão reduzida.

12.11.3.4 Destilação: o professor e o aluno

Destilação (distillation) é treinar um modelo pequeno — o **aluno** (student) — para imitar as saídas de um modelo grande — o **professor** (teacher) (Figura 10). Em vez de aprender só com os rótulos "secos" do dataset, o aluno aprende com as **distribuições de saída** do professor (as probabilidades que ele atribui a cada resposta), que carregam informação mais rica sobre *como* o professor raciocina.

Figura 10 – Destilação: o professor e o aluno



Fonte: autoria própria.

Diferença essencial para a quantização: a quantização **mantém a arquitetura** e encolhe os números; a destilação **cria um modelo de arquitetura menor** por meio de treino. As duas se combinam — é comum destilar primeiro e quantizar o aluno depois.

Quando a destilação compensa:

- Quando mesmo o modelo quantizado ainda é grande ou lento demais para o alvo — você precisa de *menos parâmetros*, não só de parâmetros menores.
- Quando você tem acesso a dados representativos da tarefa e a computação para treinar.
- Quando o uso é um **domínio restrito**: o aluno não precisa saber tudo o que o professor sabe, só o recorte relevante — e aí um modelo muito menor mantém qualidade surpreendente.

Quando não compensa: para um hackathon com prazo curto, treinar um aluno do zero raramente cabe no cronograma. O movimento realista é usar modelos pequenos **já destilados e publicados** pela comunidade/fabricantes — colhendo o benefício sem pagar o treino.

12.11.3.5 Outras otimizações: pruning e LoRA/adaptadores

Pruning (poda) parte de uma observação: depois do treino, muitos pesos contribuem quase nada para o resultado. O pruning **remove** esses pesos (ou neurônios/blocos inteiros, no *structured pruning*), seguido em geral de um fine-tuning curto para recuperar a qualidade. Ressalva importante: zerar pesos avulsos só vira ganho real de velocidade se o runtime souber explorar essa esparsidade — remoção estruturada (blocos, canais) tende a se traduzir em ganho prático com mais facilidade.

LoRA (Low-Rank Adaptation) e adaptadores atacam um problema diferente: não reduzem o modelo, reduzem o custo de **especializá-lo**. Em vez de refinar bilhões de parâmetros, o LoRA congela o modelo base e treina apenas pequenas matrizes de baixo posto acopladas a ele — o "adaptador" resultante tem poucos megabytes. No edge, isso permite distribuir **um** modelo base e vários adaptadores leves, um por tarefa, em vez de vários modelos gigantes.

Nota 15: LoRA não é técnica de compressão — o modelo base continua do mesmo tamanho. Ele entra no cardápio porque resolve o custo de *personalização* e *distribuição*, que no edge (onde cada megabyte baixado e armazenado conta) é um gargalo real.

12.11.3.6 Formatos de arquivo: por que o formato não é detalhe

Um modelo treinado precisa ser salvo em algum **formato de arquivo** — e essa escolha não é burocracia: o formato determina **qual runtime consegue executar o modelo** e, por consequência, **em qual hardware ele roda com eficiência**. É como codec de vídeo: o mesmo filme pode existir em vários formatos, mas só alguns têm decodificação acelerada por hardware na sua TV. Modelo em formato errado para o alvo = ou não roda, ou roda devagar.

Os três formatos que importam para o programa:

`.tflite` / `.litedrtlm` (**ecossistema LiteRT**). LiteRT é o runtime de inferência do Google para mobile e edge — é o novo nome do TensorFlow Lite. O `.tflite` é o formato clássico para modelos convertidos (visão, áudio, classificação...), compacto e desenhado para o celular, com acesso aos aceleradores do aparelho (GPU/NPU) via *delegates* — mecanismos que despacham a execução para hardware especializado. O `.litedrtlm` é o formato voltado a modelos de linguagem no LiteRT-LM. **Cenário ideal: o alvo é o celular/Android** — exatamente o caso do app companion.

GGUF. Formato do ecossistema ggml/llama.cpp, voltado a **LLMs quantizados rodando em CPU (com offload opcional para GPU) de desktop/notebook**. Um arquivo GGUF é autossuficiente: carrega pesos quantizados, tokenizer e metadados juntos — baixou, rodou. **Cenário ideal: rodar um modelo de linguagem local no notebook do hackathon**, para prototipagem e ferramentas de apoio do time.

ONNX. Formato **interoperável**: uma representação aberta do grafo do modelo que diversos frameworks exportam e diversos runtimes executam. O ONNX Runtime usa *execution providers* para despachar a execução em CPUs, GPUs e alguns aceleradores, em desktop, servidor e mobile. **Cenário ideal: quando você**

quer flexibilidade — treinar em um framework, executar em outro ambiente, ou manter portabilidade entre alvos diferentes.

Ideia-chave: o mesmo modelo pode existir em vários formatos. A pergunta certa nunca é "qual formato é melhor?", e sim **"onde este modelo vai rodar (celular × notebook) e qual runtime vai executá-lo?"** — a resposta define o formato (Tabela 9).

Tabela 9 – Formatos de modelos, runtimes típicos e cenários ideais de execução

Formato	Runtime típico	Hardware / cenário ideal
<code>.tflite</code>	LiteRT	Celular/Android (CPU, GPU e NPU via delegates) — modelos de visão, áudio e classificação no app companion
<code>.litemlm</code>	LiteRT-LM	LLMs compactos no celular/edge
GGUF	llama.cpp e derivados	CPU (e GPU) do notebook — LLM local para prototipagem no hackathon
ONNX	ONNX Runtime	Portátil entre desktop, servidor e alguns aceleradores — quando a flexibilidade importa

Fonte: autoria própria.

Na prática: a divisão natural no hackathon: o modelo que processa os frames dos óculos vai em `.tflite` dentro do app Android; se o time quiser um LLM auxiliar durante o desenvolvimento (gerar dados de teste, prototipar prompts), um GGUF quantizado rodando no notebook resolve sem custo de API.

12.11.3.7 Onde roda: CPU × GPU × NPU × TPU

O formato define o runtime; o runtime despacha para o hardware. Conheça os quatro tipos de processador (Tabela 10):

Tabela 10 – Comparação entre CPU, GPU, NPU e TPU quanto às características, funções e cenários de uso

Processador	O que é	Para que serve	Quando usar
CPU	Processador de propósito geral: poucos núcleos, muito flexíveis	Qualquer código; inferência de modelos pequenos/quantizados	Sempre disponível — é o <i>fallback</i> universal; no notebook, é o alvo padrão de LLMs em GGUF

GPU	Milhares de núcleos simples em paralelo	Operações matriciais massivas — modelos de visão, redes grandes	Quando o modelo é paralelizável e o ganho compensa o custo de energia; no Android, via delegate de GPU
NPU	Acelerador neural dedicado dentro do SoC do celular	Inferência de redes neurais (especialmente quantizadas) com máxima eficiência energética	Inferência frequente/contínua no celular, quando bateria e calor importam
TPU	ASIC do Google para redes neurais	Treino e inferência em escala (datacenter); variantes edge aparecem em alguns dispositivos Google	No contexto do hackathon, é o menos relevante — vocês não controlam esse hardware

Fonte: autoria própria.

A intuição: CPU é o canivete suíço, GPU é a linha de produção, NPU é a máquina construída para uma única tarefa — e por isso a executa gastando muito menos energia por inferência.

Verificar: o acesso à NPU no Android está em transição — a NNAPI (API clássica de aceleração neural do Android) foi descontinuada, e o caminho recomendado passa pelos delegates/aceleradores do próprio runtime (LiteRT) e SDKs de fabricantes. Cheque a documentação oficial do LiteRT sobre aceleração antes de contar com a NPU no aparelho do time.

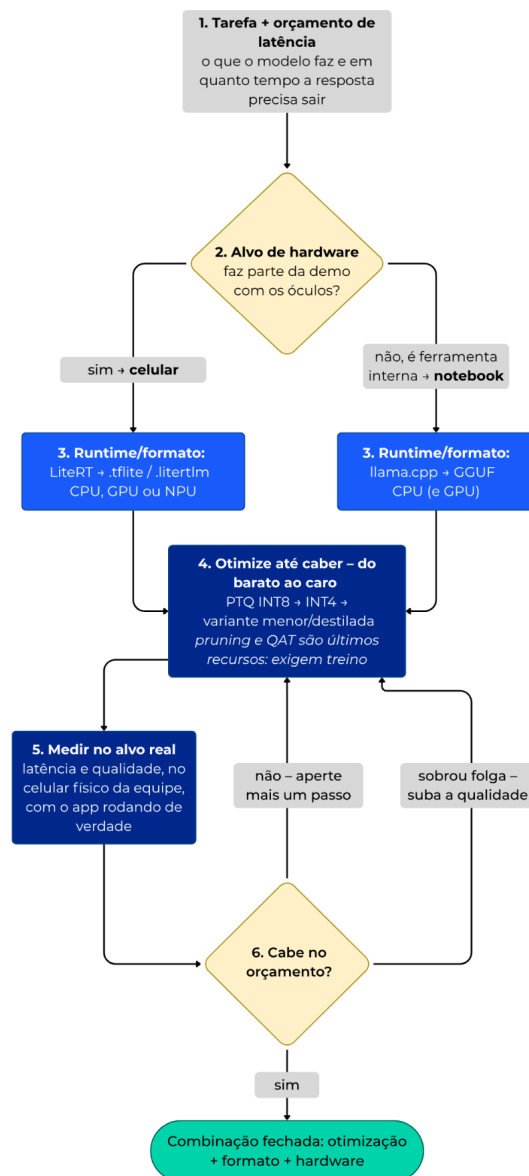
Celular × notebook — a diferença que muda a estratégia. O celular tem RAM menor e compartilhada, envelope térmico apertado e bateria; favorece modelos pequenos, quantizados, em `.tflite`, de preferência acelerados por NPU/GPU. O notebook do hackathon tem mais RAM, refrigeração ativa e tomada: **aguenta modelos maiores e formatos mais pesados** — um LLM 7B quantizado em GGUF roda tranquilamente na CPU. Regra prática: o que o usuário final vê na demo com os óculos precisa rodar no **celular**; o notebook é o laboratório do time.

12.11.3.8 O menu de alternativas: escolhendo a combinação

Nenhuma técnica deste tópico vive sozinha — o trabalho do time é **compor**: otimização + formato + hardware, com a latência como critério de desempate (Figura 11). O raciocínio, em ordem:

1. **Tarefa e orçamento de latência.** O que o modelo faz e em quanto tempo a resposta precisa sair? Resposta por voz ao usuário dos óculos = orçamento apertado.
2. **Alvo de hardware.** Faz parte da demo com os óculos? → celular. É ferramenta interna do time? → notebook.
3. **Runtime e formato decorrem do alvo.** Celular → LiteRT → `.tflite/.litemlm`. Notebook + LLM → llama.cpp → GGUF. Múltiplos ambientes → ONNX.
4. **Otimize até caber, do barato ao caro.** Comece com PTQ INT8 (custo quase zero). Não coube ou está lento? INT4. Ainda não? Procure uma variante menor/destilada do modelo. Pruning e QAT são os últimos recursos — exigem treino.
5. **Meça no alvo real.** Latência e qualidade, no celular físico do time, com o app rodando de verdade.
6. **Itere.** Se sobrou folga, dá para subir a qualidade (modelo maior, mais precisão); se faltou, aperte mais um passo do item 4.

Figura 11 – Funil de decisão: otimização + formato + hardware



Fonte: autoria própria.

O tópico 1.7 aplica esse menu às tarefas de visão computacional (detecção, classificação, OCR nos frames da câmera dos óculos) — este tópico dá o vocabulário; o próximo, a aplicação.

12.11.4 Na prática

Este tópico é conceitual, então a prática aqui é **fazer contas e tomar decisões com números** — a habilidade que o time vai usar no hackathon antes de escrever qualquer linha de código de inferência.

12.11.4.1 Passo 1 — Estime a memória antes de baixar

Antes de baixar qualquer modelo, aplique a fórmula da seção 4.2. Suponha um celular com 8 GB de RAM (dos quais o sistema e outros apps já consomem uma boa parte) e três candidatos de LLM compacto (Tabela 11):

Tabela 11 – Estimativa de memória dos modelos em diferentes níveis de quantização e sua compatibilidade com dispositivos móveis

Candidato	FP16	INT8	INT4	Cabe no celular?
0,5B	~1 GB	~0,5 GB	~0,25 GB	Sim, com folga
3B	~6 GB	~3 GB	~1,5 GB	Só quantizado (INT8/INT4)
7B	~14 GB	~7 GB	~3,5 GB	Arriscado até em INT4 — melhor no notebook

Fonte: autoria própria.

Dez segundos de aritmética eliminam becos sem saída de horas.

12.11.4.2 Passo 2 — O efeito da quantização em um modelo de visão

Números **ilustrativos** para um classificador de imagens da classe MobileNet rodando no celular (Tabela 12):

Tabela 12 – Comparação entre as versões FP32 e INT8 de um modelo MobileNet

Versão	Tamanho	Latência por inferência (CPU)	Acurácia
FP32	~16 MB	~90 ms (<i>ilustrativo</i>)	referência
INT8 (PTQ)	~4 MB	~30–45 ms (<i>ilustrativo</i>)	queda tipicamente pequena

Fonte: autoria própria.

A redução de tamanho para $\sim\frac{1}{4}$ é aritmética (4 bytes $\rightarrow$ 1 byte por peso); o ganho de latência de 2–3 $\times$ em CPU é a ordem de grandeza tipicamente citada pela documentação de quantização — o número real depende do chip, do modelo e do delegate, e **só a medição no aparelho do time responde**.

A 30 ms por frame, dá para analisar vários frames por segundo vindos dos óculos; a 90 ms, a experiência já engasga se houver mais etapas no pipeline.

12.11.4.3 Passo 3 — O LLM do notebook em GGUF

O mesmo modelo em GGUF com quantização de ~4 bits cai para a faixa de ~4 GB e roda na CPU via llama.cpp, com **velocidade de geração dependente do hardware** (avalie na sua máquina).

Na prática: monte o orçamento de latência de ponta a ponta da sua demo: captura nos óculos → transferência Bluetooth → pré-processamento → inferência → síntese de voz → áudio nos alto-falantes. A **inferência** é só uma fatia do total — e o Bluetooth já consome parte do orçamento antes de o modelo ver o primeiro byte. É por isso que a escolha INT8 + acelerador, que parece "otimização prematura", é na verdade o que sobra de margem para o resto do pipeline.

12.11.4.4 Passo 4 — Um gostinho do "como" no Android

O carregamento de um `.tflite` no app companion, só para visualizar onde as decisões deste tópico se materializam (o pipeline completo de visão é assunto do tópico 1.7):

► **Código 1.6-01** — build.gradle.kts (módulo) — dependência do runtime LiteRT — código completo no notebook companion (seção 1.6.4.4).

Repare que a decisão de quantização já aconteceu **antes** do app: o arquivo nos assets já é o INT8. O código Android apenas colhe o resultado.

Atenção! Dois erros comuns nesta etapa:

- 1) **Medir latência no emulador** — o emulador não tem a NPU/GPU do aparelho real e distorce tudo; meça sempre em um celular físico.
- 2) **Assumir que INT8 é sempre mais rápido em qualquer processador** — delegates de GPU historicamente preferem FP16/FP32, e um modelo INT8 pode acabar caindo de volta na CPU. Combine a precisão escolhida com o alvo de execução, e confirme medindo.

12.11.5 Resumo / cheatsheet

- Edge AI = inferência no próprio dispositivo: ganha latência, privacidade, offline e custo; paga em memória, calor e energia.

- Memória dos pesos $\approx$ parâmetros $\times$ bytes por parâmetro: FP32 = 4 B, FP16 = 2 B, INT8 = 1 B, INT4 = 0,5 B.
- Quantização: PTQ é barata (sem retreino, boa em INT8); QAT recupera qualidade em bits baixos, mas exige treinar.
- Destilação cria um modelo **menor em arquitetura** treinado para imitar um maior; combina com quantização; no hackathon, use alunos já publicados.
- Pruning remove pesos pouco importantes (ganho depende do runtime); LoRA especializa com adaptadores de poucos MB sem mexer no modelo base.
- Formato define runtime e hardware: `.tflite/.litemlm` $\rightarrow$ LiteRT $\rightarrow$ celular; GGUF $\rightarrow$ llama.cpp $\rightarrow$ notebook; ONNX $\rightarrow$ portátil entre ambientes.
- CPU = fallback universal; GPU = paralelismo bruto; NPU = eficiência energética no celular; TPU = ASIC do Google, pouco relevante no hackathon.
- Receita: alvo de hardware $\rightarrow$ formato do runtime $\rightarrow$ INT8 primeiro $\rightarrow$ medir latência e qualidade **no aparelho real** $\rightarrow$ iterar.

12.11.6 Referências

- LiteRT — Google AI Edge. Disponível em: <https://ai.google.dev/edge/litert>. Visão geral do runtime de inferência para mobile/edge (ex-TensorFlow Lite), conversão de modelos e aceleração por delegates.
- Post-training quantization — Google AI Edge. Disponível em: https://ai.google.dev/edge/litert/models/post_training_quantization. Guia oficial de quantização pós-treinamento: opções (dynamic range, INT8 completo, FP16) e trade-offs.
- LiteRT-LM — repositório oficial (google-ai-edge). Disponível em: <https://github.com/google-ai-edge/LiteRT-LM>. Runtime e formato para executar modelos de linguagem no ecossistema LiteRT.

- llama.cpp — repositório oficial (ggml-org). Disponível em: <https://github.com/ggml-org/llama.cpp>. Runtime de LLMs quantizados em CPU/GPU de desktop e notebook; documentação do ecossistema GGUF.
- Especificação GGUF — ggml (ggml-org). Disponível em: <https://github.com/ggml-org/ggml/blob/master/docs/gguf.md>. Especificação técnica do formato GGUF (estrutura do arquivo, metadados, tipos de quantização).
- ONNX. Disponível em: <https://onnx.ai>. Especificação do formato aberto e ecossistema de ferramentas de interoperabilidade.
- ONNX Runtime — Microsoft. Disponível em: <https://onnxruntime.ai>. Runtime multiplataforma para modelos ONNX e seus execution providers (CPU, GPU, aceleradores).

12.11.6.1 Para ir além



TensorFlow Model Optimization Toolkit. Disponível em: https://www.tensorflow.org/model_optimization. Ferramentas oficiais de quantização (PTQ e QAT) e pruning, com tutoriais.



LLM Inference — MediaPipe / Google AI Edge. Disponível em: https://ai.google.dev/edge/mediapipe/solutions/genai/llm_inference. Execução de LLMs on-device no Android, incluindo suporte a adaptadores LoRA.



AI no Android — Android Developers. Disponível em: <https://developer.android.com/ai>. Panorama oficial das opções de IA on-device na plataforma Android.

12.12 Visão computacional: tarefas e como levá-las ao edge

12.12.1 Objetivos de aprendizagem

Ao final, você será capaz de:

- Explicar as principais tarefas de visão computacional (classificação, detecção, segmentação, OCR, landmarks, captioning e rastreamento) e dar exemplos de uso com a câmera em primeira pessoa dos óculos.
- Comparar o custo computacional dessas tarefas e avaliar quais são viáveis para rodar on-device.
- Explicar o que são transfer learning e fine-tuning e como aplicá-los a um modelo YOLO com o dataset da sua equipe.
- Implementar o caminho completo de uma detecção para o edge: fine-tuning → quantização/conversão → execução no celular em CPU ou GPU.
- Escolher entre ML Kit, MediaPipe e LiteRT como caminho de implementação para uma tarefa de visão.

12.12.2 Pré-requisitos

- Tópico 1.6 (Edge AI): quantização, otimização e formatos de modelo — a teoria de lá é usada aqui sem reexplicação.
- Tópico 1.2 (Coroutines e Flow) e 1.3 (Fundamentos de Android), para os trechos de código.
- Android Studio atualizado e um celular Android físico para testar inferência real (o emulador não reflete o desempenho de CPU/GPU).
- Python 3.10+ no computador (o fine-tuning acontece fora do Android). Recomendamos gerenciar o ambiente com `uv`.

12.12.3 Conceito / fundamentação

Visão computacional é o conjunto de técnicas que permite a um programa extrair informação de imagens e vídeo: o que aparece, onde aparece, que texto está escrito, que gesto uma mão faz. No contexto do programa, a fonte de imagens é a câmera ultra-wide de 12 MP dos óculos — uma câmera em primeira pessoa, que vê o que o usuário vê. E, como os óculos não têm display, o resultado de qualquer tarefa de visão precisa virar outra coisa: áudio nos alto-falantes open-ear, uma ação no app, um registro. Essa combinação — entrada visual em primeira pessoa, saída por voz — é o que torna a escolha da tarefa certa tão importante para a sua ideia de hackathon.

Esta seção funciona como um catálogo: para cada tarefa, o que ela resolve, um exemplo nos óculos e o peso esperado ao rodar on-device (no celular pareado, como definido na arquitetura do programa).

12.12.3.1 Classificação de imagem

O que resolve: atribui um ou mais rótulos à imagem inteira ("é um prato de comida", "é uma planta", "é um documento"). Não diz onde o objeto está — só o que domina a cena.

Exemplo nos óculos: o usuário olha para um prato e o app identifica o tipo de refeição; ou detecta que a cena é "ambiente de obra" para ativar um modo específico do app.

Peso on-device: leve. Modelos como MobileNet foram desenhados exatamente para isso e rodam em poucos milissegundos por frame em CPU de celular.

12.12.3.2 Detecção de objetos

O que resolve: encontra o que há na imagem **e onde:** para cada objeto, uma caixa delimitadora (bounding box), uma classe e uma confiança. É a tarefa mais versátil do catálogo — muita ideia de hackathon se reduz a "detectar X no campo de visão".

Exemplo nos óculos: identificar produtos numa prateleira, detectar se um trabalhador está sem capacete, avisar (por áudio) sobre um obstáculo à frente para uma pessoa com deficiência visual.

Peso on-device: médio, dependendo do modelo. Variantes compactas (como as YOLO nano, detalhadas adiante) rodam em tempo real no celular; variantes grandes já pedem GPU.

12.12.3.3 Segmentação

O que resolve: classifica a imagem pixel a pixel, gerando uma máscara. Na segmentação semântica, cada pixel recebe uma classe ("chão", "pessoa", "céu"); na segmentação de instâncias, cada objeto ganha a própria máscara. É a resposta mais rica espacialmente — e a mais cara.

Exemplo nos óculos: separar precisamente um objeto do fundo (para medir área ocupada, por exemplo) ou identificar a região "caminho livre" no chão.

Peso on-device: médio a pesado. Existem modelos móveis de segmentação (o MediaPipe oferece alguns prontos), mas o custo por frame é bem maior que o da detecção. Use quando a caixa da detecção não basta.

12.12.3.4 Reconhecimento de texto (OCR)

O que resolve: localiza e transcreve texto presente na imagem: placas, rótulos, cardápios, documentos.

Exemplo nos óculos: o usuário olha para um cardápio ou para o rótulo de um remédio e o app lê o conteúdo em voz alta (a síntese de voz será vista no tópico 1.8).

Peso on-device: médio. O ML Kit oferece OCR on-device pronto, com bom desempenho em texto impresso; texto manuscrito ou em ângulos extremos degrada o resultado. Com a câmera em primeira pessoa, enquadramento e estabilidade importam: texto pequeno e distante é o principal inimigo.

12.12.3.5 Landmarks (rosto, mãos e pose)

O que resolve: em vez de caixas, retorna pontos-chave: os 468 pontos da malha facial, as juntas dos dedos, as articulações do corpo. Com esses pontos dá para inferir gestos, expressões e posturas.

Exemplo nos óculos: como a câmera é em primeira pessoa, as mãos do próprio usuário aparecem no campo de visão — dá para reconhecer gestos das mãos como comandos. Para pose, o alvo é outra pessoa: um app de treino que observa a postura do aluno e corrige por áudio.

Peso on-device: leve a médio. Os pipelines de landmarks do MediaPipe foram criados para tempo real em celular.

12.12.3.6 Captioning / descrição de imagem

O que resolve: gera uma frase em linguagem natural descrevendo a cena ("uma pessoa segurando uma caneca vermelha ao lado de um notebook"). Diferente das tarefas anteriores, exige um modelo multimodal (visão + linguagem).

Exemplo nos óculos: descrever a cena completa em voz alta para uma pessoa com deficiência visual — o caso de uso mais direto da combinação câmera + alto-falantes.

Peso on-device: pesado — é a tarefa mais cara do catálogo. Modelos multimodais compactos para dispositivo existem e evoluem rápido, mas espere latência de segundos, não milissegundos. Para o hackathon, trate captioning como recurso pontual (descrever sob demanda), não como algo contínuo por frame.

12.12.3.7 Rastreamento (tracking)

O que resolve: mantém a identidade de um objeto ao longo dos *frames*: "a pessoa da caixa 3 no *frame* anterior é a mesma da caixa 1 agora". Normalmente é uma camada sobre a detecção, não um modelo separado.

Exemplo nos óculos: seguir um objeto de interesse enquanto o usuário move a cabeça (e a câmera junto), ou contar objetos que cruzam o campo de visão sem contar duas vezes.

Peso on-device: custo adicional moderado sobre a detecção — *trackers* clássicos são baratos; o desafio nos óculos é que a câmera se move o tempo todo com a cabeça do usuário, o que exige detecção frequente para reancorar o rastreamento.

12.12.3.8 O catálogo em uma tabela

As tarefas estão descritas na Tabela 13.

Tabela 13 – Catálogo de tarefas de visão computacional, resultados esperados e caminhos de implementação on-device

Tarefa	O que entrega	Peso on-device	Caminho típico
Classificação	rótulo(s) da imagem inteira	leve	ML Kit (image labeling) ou modelo próprio

Detecção de objetos	caixas + classes + confiança	leve a médio	YOLO via LiteRT; ML Kit/MediaPipe
Segmentação	máscara pixel a pixel	médio a pesado	MediaPipe; modelo próprio
OCR	texto localizado e transcrito	médio	ML Kit (text recognition)
Landmarks	pontos-chave de rosto/mãos/pose	leve a médio	MediaPipe; ML Kit
Captioning	frase descrevendo a cena	pesado	modelo multimodal
Rastreamento	identidade entre frames	adicional sobre detecção	camada sobre o detector

Fonte: autoria própria.

Na prática: ao escolher a tarefa para a sua ideia, pense no ciclo completo dos óculos: o frame chega ao celular por streaming Bluetooth (como será visto no tópico 2.5), o modelo roda no celular e o resultado precisa virar áudio útil nos alto-falantes. Uma tarefa leve rodando de forma contínua (detecção) e uma pesada rodando sob demanda (captioning quando o usuário pede) podem conviver no mesmo app — essa é uma composição comum e eficiente.

Atenção! "Roda no meu notebook" não significa "roda no celular". A viabilidade on-device se decide pelo tamanho do modelo, pela latência por frame no aparelho-alvo e pelo consumo de bateria (revisite o tópico 1.5). Sempre valide no dispositivo físico o quanto antes.

12.12.3.9 YOLO: a família de referência para detecção em tempo real

YOLO ("You Only Look Once") é uma das famílias de modelos mais famosas e eficientes para detecção de objetos em tempo real. A ideia que dá nome à família: em vez de analisar a imagem em várias etapas, o modelo faz uma única passada pela rede e já produz todas as caixas e classes. Esse desenho o torna rápido — e é por isso que ele se encaixa tão bem no cenário dos óculos, em que *frames* de uma câmera em primeira pessoa chegam continuamente e precisam de resposta com baixa latência.

A implementação mais usada hoje é a da Ultralytics, que mantém as gerações recentes da família (YOLOv8, YOLO11 e, mais recentemente, YOLO26) com uma API Python unificada de treino e exportação.

12.12.3.9.1 Tamanhos e variantes

Cada geração YOLO vem em variantes de tamanhos diferentes — tipicamente **n** (nano), **s** (small), **m** (medium), **l** (large) e **x** (extra). Todas fazem a mesma tarefa; o que muda é o trade-off (Tabela 14):

Tabela 14 – Variantes do YOLO: precisão, velocidade e adequação ao edge

Variante	Precisão	Velocidade / leveza	Candidata a edge?
n (nano)	menor	máxima	sim — primeira escolha para CPU de celular
s (small)	boa	alta	sim — boa com GPU ou CPU forte
m (medium)	melhor	média	só com GPU/acelerador
l / x	máxima	baixa	raramente; pensadas para servidor/desktop

Fonte: autoria própria.

A regra prática para o programa: comece pela **nano**, meça a latência no celular-alvo e só suba de variante se a precisão for insuficiente **e** houver folga de desempenho (por exemplo, com GPU disponível — ver adiante).

12.12.3.9.2 Transfer learning: não treine do zero

Os modelos YOLO pré-treinados aprenderam com datasets grandes de uso geral (como o COCO, com 80 classes do cotidiano). Nesse processo, as camadas iniciais da rede aprenderam a enxergar coisas universais: bordas, texturas, formas, partes de objetos. **Transfer learning** é aproveitar todo esse conhecimento como ponto de partida em vez de começar com uma rede "em branco". A analogia: contratar alguém que já sabe ler e escrever e ensiná-lo o vocabulário técnico da sua área — em vez de alfabetizá-lo do zero.

12.12.3.9.3 Fine-tuning: o modelo aprende as SUAS classes

Fine-tuning é a aplicação prática do transfer learning: você pega o modelo pré-treinado e o re-treina com um dataset próprio, contendo as classes que interessam à sua equipe (os objetos da sua ideia de hackathon: um tipo de equipamento, um produto específico, uma sinalização). Como o modelo já "sabe ver", ele precisa de **pouco dado e pouco tempo** para aprender as classes novas — algumas centenas de imagens anotadas e algumas dezenas de épocas de treino

costumam bastar para um protótipo funcional. Isso é perfeitamente viável dentro do cronograma do programa, inclusive usando GPUs gratuitas de serviços como o Google Colab.

O que a equipe precisa produzir é o dataset: imagens dos objetos-alvo (idealmente capturadas em condições parecidas com as reais — primeira pessoa, distâncias e iluminações variadas) e as anotações de caixa para cada objeto.

Nota 16: o resultado do fine-tuning é um modelo em precisão cheia (`float32`), no formato de treino (`.pt`, do PyTorch). Ele ainda **não** está pronto para o celular: como visto no 1.6, falta o passo de otimização/quantização e a conversão para um formato de execução móvel (como `.tflite`). Fine-tuning e otimização são etapas distintas e ambas necessárias — a seção "Na prática" percorre as duas.

12.12.3.10 Como levar uma tarefa ao edge: três caminhos

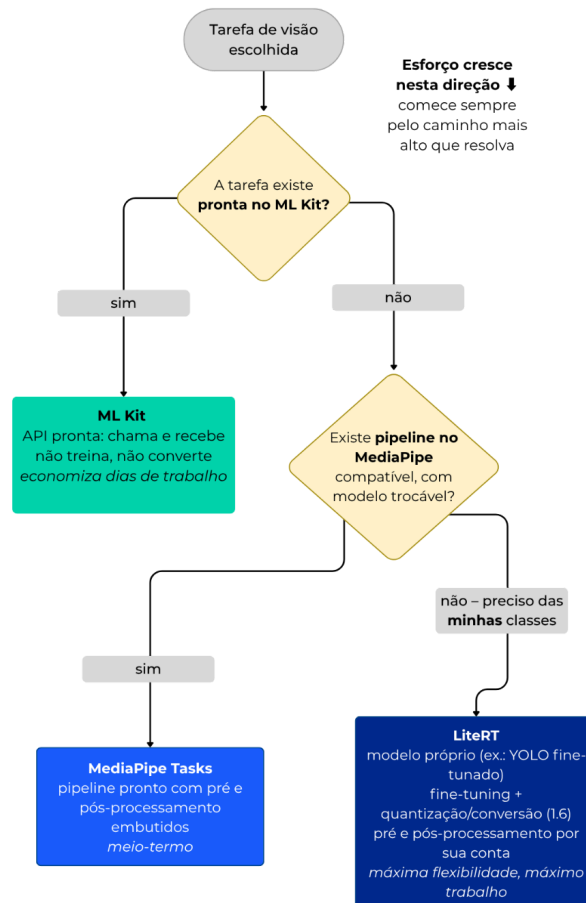
Com a tarefa escolhida, há três caminhos de implementação no Android — do mais pronto ao mais flexível (Figura 12):

1. **ML Kit** — APIs prontas do Google para tarefas comuns (OCR, detecção genérica de objetos, landmarks de rosto e pose, image labeling, entre outras). Você não treina nem converte nada: chama a API e recebe o resultado. É o caminho mais rápido quando a tarefa padrão resolve o seu problema. Algumas APIs aceitam plugar um modelo customizado, mas o forte do ML Kit é a prateleira pronta.
2. **MediaPipe Tasks** — pipelines prontos de visão (detecção, segmentação, landmarks de mãos/rosto/pose) com pré e pós-processamento embutidos, permitindo trocar o modelo por um seu quando suportado. Meio-termo entre prateleira e customização.
3. **LiteRT** — o runtime de inferência do Google AI Edge (evolução do TensorFlow Lite, visto no 1.6). Aqui você roda **qualquer** modelo seu — como o YOLO com fine-tuning — e cuida do pré e pós-processamento. Máxima flexibilidade, máximo trabalho.

A decisão prática: **se existe API pronta no ML Kit para a sua tarefa, comece por ela** (você economiza dias de trabalho); se a tarefa existe no MediaPipe com possibilidade de modelo próprio, avalie-o; se você precisa de

classes/comportamento que só um modelo seu entrega — o caso do YOLO fine-tunado — o caminho é o LiteRT.

Figura 12 – Três caminhos para levar uma tarefa de visão ao edge



Fonte: autoria própria.

12.12.3.11 Onde o modelo roda no celular: CPU, GPU e aceleradores

O celular pareado tem mais de um processador capaz de rodar inferência:

- **CPU** — o caminho universal: funciona em qualquer aparelho, sem verificação prévia. O LiteRT usa por padrão kernels otimizados (XNNPack) e várias threads. Para modelos pequenos (uma YOLO nano quantizada), a CPU costuma bastar.
- **GPU** — via *delegate* (um plugin do runtime que desvia a execução para outro processador). Em aparelhos compatíveis, reduz a latência principalmente de modelos **maiores**, onde há mais cálculo para paralelizar.

Tem custos: inicialização mais lenta, nem todo aparelho suporta, e nem toda operação do modelo é aceita (o que não for cai de volta na CPU).

- **NPU/DSP e outros aceleradores** — chips dedicados a IA presentes em muitos aparelhos, acessados por delegates específicos de fabricante. A antiga via genérica (NNAPI) foi descontinuada pelo Android. Para o programa, trate CPU e GPU como os dois alvos realistas.

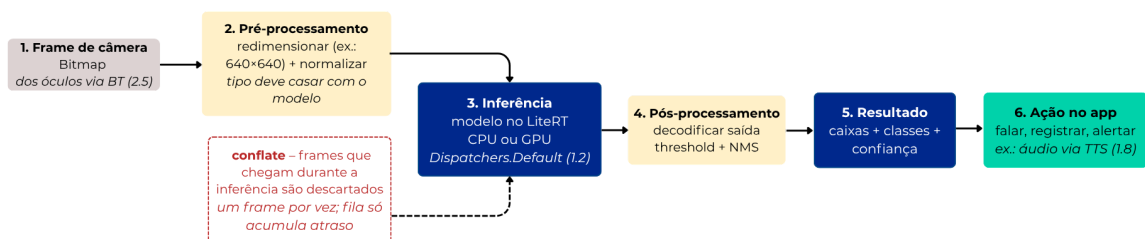
Essa escolha **influencia a variante do modelo**: uma YOLO nano na CPU e uma YOLO small/medium na GPU podem ter latências parecidas — a segunda com mais precisão. Se o aparelho-alvo da equipe tem GPU compatível, vale testar uma variante acima; se o app precisa rodar em qualquer aparelho, projete para a nano na CPU e trate a GPU como bônus.

Nota 17: GPU não é sinônimo de "sempre mais rápido". Para modelos muito pequenos, o overhead de preparar a execução na GPU pode comer o ganho. A única resposta confiável é medir a latência no aparelho-alvo, nos dois modos.

12.12.3.12 Do frame ao resultado: o pipeline

Independentemente da tarefa e do caminho escolhidos, o fluxo de execução é sempre o mesmo (Figura 13):

Figura 13 – Do frame ao resultado: o pipeline universal de visão



Fonte: autoria própria.

1. **Frame** — uma imagem individual do fluxo da câmera. Aqui tratamos o frame como um `Bitmap` genérico; a captura real vinda dos óculos será vista no tópico 2.5.
2. **Pré-processamento** — o modelo espera entrada em tamanho e formato fixos (ex.: 640×640 pixels, valores normalizados). Todo frame precisa ser convertido antes.

3. **Inferência** — uma passada do modelo sobre a entrada. É o passo cujo custo você negociou ao escolher tarefa, variante e processador.
4. **Pós-processamento** — a saída crua do modelo vira resultado útil: no caso da detecção, decodificar as caixas, descartar as de baixa confiança (threshold) e remover caixas duplicadas sobre o mesmo objeto (NMS — non-maximum suppression).
5. **Resultado** → **ação** — o app decide o que fazer: falar, registrar, alertar.

Como a câmera gera frames mais rápido do que o modelo processa, o pipeline processa um frame por vez e **descarta os que chegam durante a inferência** — estratégia vista com Flow no 1.2 (`conflate`). Processar todo frame em fila só acumula atraso.

12.12.4 Na prática

Vamos percorrer o fio condutor completo do tópico com um exemplo concreto: um app de segurança de obra que detecta **capacete** e **colete** no campo de visão. O caminho tem três etapas: fine-tuning (Python, no computador) → quantização/conversão (Python) → execução no celular (Kotlin, CPU ou GPU).

12.12.4.1 Etapa 1 — Fine-tuning do YOLO com o dataset da equipe

No computador (não no Android), crie o ambiente e instale a biblioteca da Ultralytics:

```
# ambiente isolado com uv
uv venv
uv pip install ultralytics
```

Código 1.7-01 · também no notebook companion, seção 1.7.4.1

Organize o dataset no formato YOLO: imagens + um arquivo `.txt` de anotações por imagem (classe e coordenadas da caixa), e um `data.yaml` descrevendo tudo:

► **Código 1.7-02** — `dataset/data.yaml` — código completo no notebook companion (seção 1.7.4.1).

O treino em si é curto — o transfer learning acontece na primeira linha, ao carregar os pesos pré-treinados:

► **Código 1.7-03** – código completo no notebook companion (seção 1.7.4.1).

O melhor checkpoint fica salvo em `runs/detect/train/weights/best.pt`. Valide as métricas no conjunto de validação antes de seguir — não adianta otimizar um modelo ruim.

Nota 18: para anotar as imagens (desenhar as caixas), use qualquer ferramenta que exporte no formato YOLO. Capture imagens em primeira pessoa, parecidas com o que a câmera dos óculos verá: mesmas distâncias, ângulos e iluminação. Dataset parecido com a realidade vale mais do que dataset grande.

12.12.4.2 Etapa 2 — Quantização e conversão (amarrando com o 1.6)

O `best.pt` é float32 e formato PyTorch — inadequado para o celular. Aplicamos agora o que o 1.6 ensinou: quantização INT8 e conversão para `.tflite`, em uma única chamada de exportação:

► **Código 1.7-04** – código completo no notebook companion (seção 1.7.4.2).

Atenção! Não pule a validação pós-quantização. Rode o modelo quantizado sobre o conjunto de validação e compare as métricas com o float32 — como visto no 1.6, a quantização pode degradar a precisão, e é melhor descobrir isso agora do que no dispositivo.

12.12.4.3 Etapa 3 — Execução no celular com LiteRT (CPU ou GPU)

No projeto Android, adicione as dependências do LiteRT e copie o `.tflite` para `assets/`:

► **Código 1.7-05** – `build.gradle.kts` (módulo app) — código completo no notebook companion (seção 1.7.4.3).

Carregue o modelo e crie o interpretador, escolhendo CPU ou GPU conforme a compatibilidade do aparelho:

► **Código 1.7-06** – Carrega o `.tflite` de `assets/` como buffer mapeado em memória — código completo no notebook companion (seção 1.7.4.3).

É exatamente aqui que a escolha de variante volta à cena: se `isDelegateSupportedOnThisDevice` for verdadeiro no aparelho-alvo da equipe, você pode embarcar uma variante maior; se o app precisa rodar em qualquer aparelho, embarque a nano e conte só com a CPU.

12.12.4.4 Etapa 4 — Do frame ao resultado

A inferência nunca roda na main thread — é trabalho pesado de CPU (como visto no 1.2, usamos `Dispatchers.Default`):

► **Código 1.7-07** – código completo no notebook companion (seção 1.7.4.4).

`preprocess` e `postprocess` dependem do formato exato de entrada/saída do seu export: inspecione os tensores com `interpreter.getInputTensor(0)` e `getOutputTensor(0)` para confirmar dimensões e tipo. O pós-processamento da saída YOLO em TFLite (decodificação das caixas e NMS) varia conforme as opções de exportação.

Atenção! Erro comum: alimentar um modelo quantizado INT8 com dados float (ou vice-versa). O sintoma clássico é o modelo "rodar sem erro" mas devolver lixo — caixas absurdas, confianças zeradas. Antes de depurar o pipeline inteiro, cheque `interpreter.getInputTensor(0).dataType()` e garanta que o pré-processamento produz exatamente esse tipo.

Na prática: monte um orçamento de latência de ponta a ponta. No hackathon, o frame ainda vai atravessar o Bluetooth antes de chegar ao seu pipeline (streaming visto no tópico 2.5) e o resultado ainda vira fala (TTS, tópico 1.8). Se a inferência sozinha consome 500 ms, a experiência total ficará lenta demais para um aviso de segurança. Meça cada etapa separadamente — e enquanto os óculos não estão disponíveis, o Mock Device Kit (tópico 2.7) permite exercitar esse pipeline com dados simulados.

12.12.5 Resumo / cheatsheet

- O catálogo de visão: classificação (o que é), detecção (o que + onde), segmentação (pixel a pixel), OCR (texto), landmarks (pontos-chave), captioning (descrição em frase), rastreamento (identidade entre frames).
- Peso on-device cresce nesta ordem aproximada: classificação < detecção (compacta) $\approx$ landmarks < OCR < segmentação < captioning.
- YOLO = detecção em tempo real em uma única passada; variantes n/s/m/l/x trocam precisão por velocidade — a nano é a candidata natural para edge.
- Transfer learning: partir de um modelo pré-treinado em dataset geral; fine-tuning: re-treiná-lo com as classes da sua equipe — pouco dado, pouco tempo, viável no programa.
- Fine-tuning **não** encerra o trabalho: o modelo ainda passa por quantização e conversão de formato (teoria no 1.6) antes de ir para o celular.
- Três caminhos de implementação: ML Kit (tarefas prontas) $\rightarrow$ MediaPipe (pipelines com modelo trocável) $\rightarrow$ LiteRT (modelo próprio, máxima flexibilidade).
- CPU roda em qualquer aparelho; GPU (via delegate) acelera modelos maiores em aparelhos compatíveis — a escolha do processador influencia a variante do modelo.
- Pipeline universal: frame $\rightarrow$ pré-processamento $\rightarrow$ inferência $\rightarrow$ pós-processamento (threshold + NMS) $\rightarrow$ resultado $\rightarrow$ ação (nos óculos, tipicamente áudio).

12.12.6 Referências

- ML KIT / GOOGLE. Disponível em: <https://developers.google.com/ml-kit>. Catálogo das APIs prontas de visão on-device (OCR, detecção, landmarks, image labeling).
- LITERT / GOOGLE AI EDGE. Disponível em: <https://ai.google.dev/edge/litert>. Documentação do runtime de inferência: guia Android, formatos e otimização.

- GPU ACCELERATION DELEGATE / GOOGLE AI EDGE. Disponível em: <https://ai.google.dev/edge/litert/android/gpu>. Como habilitar o delegate GPU com a Interpreter API e as dependências necessárias.
- MEDIAPIPE SOLUTIONS / GOOGLE AI EDGE. Disponível em: <https://ai.google.dev/edge/mediapipe/solutions/guide>. Pipelines prontos de visão (detecção, segmentação, landmarks de mãos/rosto/pose).
- ULTRALYTICS (repositório oficial). Disponível em: <https://github.com/ultralytics/ultralytics>. Código e documentação de treino, fine-tuning e exportação dos modelos YOLO.
- EXPORT YOLO TO TFLITE / ULTRALYTICS DOCS. Disponível em: <https://docs.ultralytics.com/integrations/tflite>. Guia oficial de exportação para TFLite, incluindo quantização INT8 com dados de calibração.

12.12.6.1 Para ir além



MEDIAPIPE MODEL MAKER / GOOGLE AI EDGE. Disponível em: https://ai.google.dev/edge/mediapipe/solutions/model_maker. Transfer learning simplificado para os modelos usados nos pipelines do MediaPipe.



MODEL EXPORT / ULTRALYTICS DOCS. Disponível em: <https://docs.ultralytics.com/modes/export>. Todos os formatos e parâmetros de exportação disponíveis para os modelos YOLO.

12.13 Voz no dispositivo: STT e TTS – Módulo: 1 Kotlin/Android · CEIA e Meta – AI Glasses Brasil

12.13.1 Objetivos de aprendizagem

Ao final, você será capaz de:

- **Explicar** o que são STT e TTS e onde cada um entra no assistente de voz dos óculos Ray-Ban Meta.

- **Descrever** os pipelines de STT e de TTS etapa por etapa, diferenciando pipelines clássicos (módulos separados) de modelos end-to-end.
- **Comparar** modelos de STT (Whisper, Wav2Vec 2.0, Conformer/RNN-T, Vosk) e de TTS (Tacotron 2, FastSpeech 2, VITS, Piper) quanto à viabilidade on-device.
- **Aplicar** técnicas de processamento de voz — VAD, diarização e palavra de ativação — e escolher entre estratégias como streaming × lote e push-to-talk × sempre-escutando.
- **Implementar** STT e TTS no Android com as APIs nativas (`SpeechRecognizer` / `TextToSpeech`) e conhecer o caminho para rodar modelos on-device (Whisper tiny, Piper).

12.13.2 Pré-requisitos

- **Tópico 1.2 (Coroutines e Flow)** — a inferência de voz roda fora da main thread; usaremos `withContext` e `Dispatchers`.
- **Tópico 1.4 (Permissões em runtime)** — STT exige a permissão `RECORD_AUDIO` concedida em tempo de execução.
- **Tópico 1.6 (Edge AI)** — quantização, formatos de modelo e trade-offs de rodar IA no aparelho; não reexplicaremos isso aqui.
- **Ferramentas** — Android Studio atualizado e um **dispositivo físico** com microfone (o emulador tem limitações de captura de áudio).

12.13.3 Conceito / fundamentação

O assistente dos óculos conversa: o usuário **fala** um comando e o assistente **responde falando** (os óculos não têm display — a voz é a interface principal de saída). Este tópico cobre as duas metades dessa conversa: **STT**, que entende a fala, e **TTS**, que produz a resposta falada. É o "menu de técnicas" de voz, no mesmo espírito do que o tópico 1.7 fez para visão.

12.13.3.1 STT: transformar fala em texto

STT (Speech-to-Text), também chamado de **ASR (Automatic Speech Recognition)**, é a tarefa de converter um sinal de áudio contendo fala em texto. No nosso contexto, é a **porta de entrada** do assistente: o usuário fala perto dos óculos,

o áudio chega ao celular via Bluetooth e o STT o transcreve; o texto resultante é o que o restante do app consegue processar (interpretar o comando, alimentar um modelo de linguagem etc.).

Uma analogia útil: o STT é o "ouvido alfabetizado" do sistema — ele não entende o *significado* do que foi dito (isso é papel de outra camada), apenas registra fielmente o *que* foi dito.

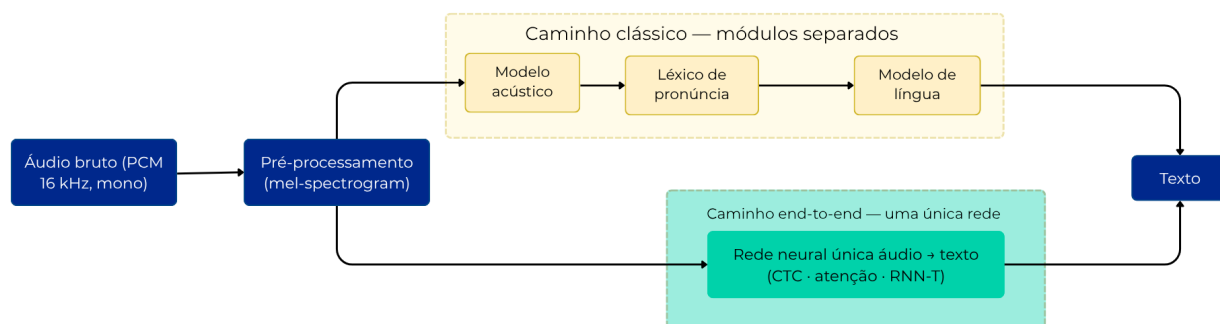
12.13.3.2 O pipeline de STT

Um sistema de STT funciona em etapas (Figura 14):

1. **Captura de áudio** — o microfone produz amostras PCM (tipicamente 16 kHz, mono, 16 bits para fala).
2. **Pré-processamento / extração de características** — o áudio bruto é convertido em uma representação mais compacta e informativa, quase sempre um **mel-spectrogram**: uma "imagem" que mostra quanta energia existe em cada faixa de frequência ao longo do tempo, usando a escala *mel*, que aproxima a percepção humana de graves e agudos. Pense nele como a "partitura" do áudio.
3. **Modelo acústico + decodificador** — uma rede neural mapeia as características em unidades de fala (caracteres, fonemas ou sub-palavras) e um decodificador monta a sequência final de texto, possivelmente com ajuda de um modelo de língua.
4. **Texto** — a transcrição, muitas vezes acompanhada de hipóteses alternativas e scores de confiança.

Clássico × end-to-end. Nos pipelines **clássicos**, as etapas são módulos separados e treinados individualmente: modelo acústico + léxico de pronúncia + modelo de língua, combinados na decodificação. Nos modelos **end-to-end**, uma única rede neural é treinada para mapear áudio → texto diretamente (com técnicas como CTC, atenção encoder-decoder ou RNN-T). End-to-end simplifica o sistema e domina o estado da arte; o clássico ainda vence em cenários de recursos muito limitados e vocabulário controlado.

Figura 14 – Comparação pipelines de STT por etapas x end-to-end



Fonte: autoria própria.

12.13.3.3 Modelos de STT e viabilidade on-device

Na Tabela 15, a seguir, você encontra a ideia central de cada modelo de STT e sua viabilidade *on-device*.

Tabela 15 – Modelos de STT e sua viabilidade *on-device*

Modelo / família	Ideia central	On-device?
Whisper (OpenAI)	Encoder-decoder Transformer, end-to-end, multilíngue (inclui pt-BR), robusto a ruído. Variantes de <i>tiny</i> (~39 M parâmetros) a <i>large</i> ; existem versões destiladas (Distil-Whisper) menores e mais rápidas.	<i>tiny</i> / <i>base</i> rodam em celular, especialmente quantizadas (ver 1.6). Processa em lote (janelas de áudio) — não nasceu para streaming.
Wav2Vec 2.0 (Meta AI)	Pré-treinamento autossupervisionado em áudio sem transcrição; fine-tuning com pouca supervisão, decodificação via CTC .	Possível com variantes pequenas quantizadas, mas é mais comum como base de pesquisa/fine-tuning do que como solução mobile pronta.
Conformer / RNN-T	Encoder Conformer (convolução + Transformer) com decodificador RNN-T , projetado para streaming com baixa latência.	Sim — é a família por trás de ASR on-device comercial (ditado de teclado, legendas ao vivo em celulares).
Vosk / Kaldi	Toolkit clássico leve e offline; o Vosk empacota modelos Kaldi para mobile com API simples.	Sim — modelos pt-BR compactos (dezenas de MB), streaming nativo, latência baixa; qualidade inferior ao Whisper em fala espontânea.

Fonte: autoria própria.

Na prática: para o hackathon, a escolha típica é **Whisper tiny ou base quantizado** para transcrever comandos curtos em lote (o usuário fala, o áudio é fechado, o modelo transcreve), ou **Vosk** se você precisar de transcrição contínua em streaming com pouquíssimo recurso. Whisper large e afins ficam fora do alcance do celular.

12.13.3.4 TTS: transformar texto em fala

TTS (Text-to-Speech) é a tarefa inversa: converter texto em áudio de fala natural. No assistente dos óculos, é a **porta de saída**: a resposta gerada pelo app vira voz e é reproduzida nos alto-falantes open-ear. Sem display, o TTS não é um "extra" — é o único canal rico de resposta ao usuário.

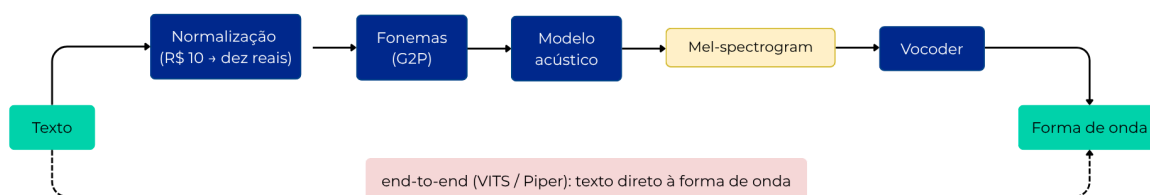
12.13.3.5 O pipeline de TTS

O pipeline clássico de síntese também funciona em etapas (Figura 15):

1. **Normalização de texto** — expandir números, siglas e símbolos para a forma falada: "R\$ 10" → "dez reais", "Dr." → "doutor".
2. **Análise linguística / fonemas** — converter grafemas em fonemas (G2P, *grapheme-to-phoneme*) e extrair marcações de prosódia (pausas, ênfase).
3. **Modelo acústico** — uma rede neural converte a sequência de fonemas em um **mel-spectrogram** (o mesmo tipo de representação usado no STT, agora como *saída* intermediária).
4. **Vocoder** — uma segunda rede converte o mel-spectrogram em **forma de onda** (as amostras de áudio finais).

End-to-end. Modelos como o **VITS** unificam tudo: uma única rede vai do texto direto à forma de onda, treinada de ponta a ponta — menos componentes para integrar e menos fontes de erro acumulado.

Figura 15 – Comparação pipelines de TTS por etapas x end-to-end



Fonte: autoria própria.

12.13.3.6 Modelos de TTS e viabilidade on-device

Na Tabela 16, a seguir, você encontra a ideia central de cada modelo de TTS e sua viabilidade *on-device*.

Tabela 16 – Análise comparativa das arquiteturas de síntese de fala e de sua viabilidade para execução *on-device*

Modelo	Ideia central	On-device?
Tacotron 2 + vocoder	Modelo acústico autorregressivo (gera o mel quadro a quadro) + vocoder. O vocoder original, WaveNet , tem alta qualidade mas é lento; HiFi-GAN gera com qualidade próxima em tempo real na CPU.	Pesado para celular na forma original; historicamente importante, hoje pouco usado em mobile.
FastSpeech 2	Não-autorregressivo : gera todos os quadros do mel em paralelo — rápido e estável (não "engole" nem repete palavras). Precisa de vocoder.	Viável com vocoder leve (HiFi-GAN), mas exige montar duas peças.
VITS	End-to-end : texto → forma de onda em um único modelo, com qualidade alta.	Variantes reduzidas são viáveis; é a base do caminho mais prático abaixo.
Piper	Sistema baseado em VITS otimizado para edge (nasceu para Raspberry Pi), modelos em ONNX, dezenas de idiomas — incluindo vozes pt-BR.	Sim — é a opção de referência para TTS neural local; se roda em Raspberry Pi, roda com folga num celular moderno.

Fonte: autoria própria.

Nota 19: o par "modelo acústico + vocoder" explica por que você verá o mel-spectrogram nos dois mundos: no STT ele é a entrada do modelo; no TTS clássico, a *saída intermediária* que o vocoder transforma em som.

12.13.3.7 Técnicas de processamento de voz

Um assistente de voz real não é só "STT + TTS". Entre o microfone e o modelo há um encadeamento de técnicas que economizam bateria, reduzem latência e melhoram a experiência.

12.13.3.7.1 VAD (Voice Activity Detection)

O VAD responde uma pergunta simples: **há fala neste trecho de áudio, ou é silêncio/ruído?** Com ele, o STT (caro) só roda quando alguém de fato está falando — o resto do tempo o áudio é descartado barato. Isso corta latência (o app sabe quando a fala *terminou* e pode fechar a janela de transcrição) e poupa bateria (ver

tópico 1.5). Exemplos: **Silero VAD** (modelo neural minúsculo, na casa de poucos MB) e **WebRTC VAD** (clássico, extremamente leve).

12.13.3.7.2 Diarização

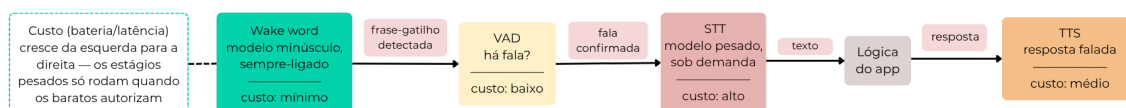
Diarização responde "**quem falou e quando**": segmenta um áudio com várias pessoas e atribui cada trecho a um locutor (Locutor A, Locutor B...). É relevante quando o caso de uso envolve **mais de uma voz** — transcrever uma reunião ou entrevista capturada pelos óculos, por exemplo. Para um assistente de comandos de um único usuário, geralmente é dispensável. É um processamento relativamente pesado, normalmente feito **em lote**, após a captura.

12.13.3.7.3 Palavra de ativação (wake word)

Uma **wake word** é uma frase-gatilho ("Ok, assistente") detectada por um modelo **minúsculo e sempre-ligado**, cujo único trabalho é reconhecer aquela frase específica. Ao detectá-la, ele "acorda" o pipeline completo (VAD → STT → resposta). A lógica é de cascata: o modelo barato roda o tempo todo; o caro, quase nunca. Exemplos: **Porcupine** e **openWakeWord** (open source) (Figura 16).

Atenção! O acionamento "**Hey Meta**" dos óculos não é acessível pelo toolkit de desenvolvimento — ele pertence ao assistente da Meta. Se a sua equipe quiser ativação por voz, precisará construir a própria wake word rodando **no celular**, sobre o áudio que chega dos óculos (o acesso ao microfone via SDK será visto no tópico 2.6).

Figura 16 – Lógica de modelo cascata com wake word



Fonte: autoria própria.

12.13.3.7.4 Estratégias de processamento

- **Streaming × em lote.** Em **streaming**, o áudio é transcrito enquanto chega — transcrições parciais aparecem em tempo real (Vosk e Conformer/RNN-T nasceram para isso). Em **lote**, o app captura a fala inteira e transcreve de uma vez (o modo natural do Whisper). Lote é mais

simples e costuma bastar para comandos curtos; streaming vale quando o usuário precisa de feedback imediato ou fala longamente.

- **Push-to-talk × sempre-escutando. Push-to-talk:** o usuário aciona explicitamente a escuta (botão no app, gesto). Simples, previsível, zero custo quando ocioso — ótimo primeiro passo no hackathon. **Sempre-escutando:** exige a cascata wake word + VAD acima e cobra seu preço em bateria (tópico 1.5), mas oferece a experiência mãos-livres que combina com óculos.
- **On-device × híbrido. On-device:** todo o processamento no celular — privacidade (o áudio da voz do usuário nunca sai do aparelho), funciona offline, latência previsível. **Híbrido:** partes pesadas na nuvem — mais qualidade, ao custo de rede, latência variável e envio de áudio para terceiros. Os checkpoints do programa valorizam IA local: se usar qualquer serviço externo, **deixe explícito**.
- **Latência e ruído.** Latência total = captura + (Bluetooth) + VAD + STT + lógica + TTS + reprodução; cada estágio soma, e o usuário percebe a soma. Quanto ao ruído, há uma boa notícia: o **array de 5 microfones dos óculos** ajuda a capturar a voz do usuário com mais limpeza do que um microfone único em ambiente barulhento — o sinal que chega ao celular via Bluetooth já é mais favorável ao STT (as características do streaming de áudio via SDK serão vistas no tópico 2.6).

12.13.3.8 Viabilidade on-device: fechando a conta

Tudo que o tópico 1.6 apresentou sobre edge AI vale aqui: modelos de voz viáveis no celular são as **variantes pequenas e quantizadas** — Whisper *tiny/base* ou Distil-Whisper para STT, Piper para TTS, Silero VAD e wake words minúsculas como estágios baratos. A conta a fechar é tripla: **memória** (o modelo precisa caber na RAM junto com o resto do app), **latência** (o usuário espera resposta em ~1–2 s de conversa natural) e **bateria** (inferência contínua drena; a cascata de ativação existe exatamente para isso).

Na prática: um pipeline de voz completo e 100% local no celular é perfeitamente alcançável no hackathon: wake word própria (ou push-to-talk) →

Silero VAD → Whisper tiny quantizado → lógica do app → Piper. Nenhum áudio sai do aparelho — argumento forte nos checkpoints de IA local e privacidade.

12.13.4 Na prática

Vamos implementar as duas pontas no Android: primeiro com as **APIs nativas** (rápido de montar, ótimo para prototipar) e depois o caminho com **modelos on-device** (controle total e privacidade garantida). Em cada abordagem, indicamos explicitamente se o processamento é local ou usa serviço.

12.13.4.1 Passo 0 — Permissão de microfone

STT exige `RECORD_AUDIO`, uma permissão *dangerous* que deve ser solicitada em runtime, como visto no tópico 1.4. No `AndroidManifest.xml`:

```
<uses-permission  
  android:name="android.permission.RECORD_AUDIO" />
```

Código 1.8-01 · também no notebook companion, seção 1.8.4.1

E no app, antes de iniciar qualquer captura, confirme a concessão (fluxo do 1.4). O `TextToSpeech` **não** exige permissão.

12.13.4.2 STT com a API nativa: `SpeechRecognizer`

O Android oferece reconhecimento de fala pronto via `android.speech.SpeechRecognizer`, que delega o trabalho a um **serviço de reconhecimento** instalado no aparelho (em geral, o da Google).

► **Código 1.8-02** – código completo no notebook companion (seção 1.8.4.2).

On-device ou serviço? Por padrão, o serviço de reconhecimento **pode enviar o áudio a servidores** — depende do fornecedor do serviço e do aparelho. Para garantir processamento local você tem duas alavancas: `EXTRA_PREFER_OFFLINE` (API 23+, apenas *prefere* o modo offline) e, a partir do **API 31**, `SpeechRecognizer.createOnDeviceSpeechRecognizer()` combinado com `isOnDeviceRecognitionAvailable()`, que usa

exclusivamente o reconhecedor local do aparelho (exige que o pacote de idioma pt-BR esteja baixado no dispositivo).

Atenção! Erros comuns com `SpeechRecognizer`:

- 1) criar ou chamar o recognizer fora da main thread — a API exige a thread principal;
- 2) esquecer `destroy()` e vaziar a conexão com o serviço;
- 3) tratar `ERROR_NO_MATCH` como falha grave — ele só significa "não entendi nada", situação normal que merece um simples "pode repetir?";
- 4) iniciar a escuta sem a permissão `RECORD_AUDIO` concedida, o que gera `ERROR_INSUFFICIENT_PERMISSIONS`.

12.13.4.3 TTS com a API nativa: `TextToSpeech`

A síntese nativa usa a **engine de TTS instalada** no aparelho (em geral, a da Google). A inicialização é **assíncrona** — este é o detalhe que mais derruba iniciantes:

► **Código 1.8-03** — código completo no notebook companion (seção 1.8.4.3).

On-device ou serviço? A engine da Google sintetiza **localmente** quando a voz do idioma está baixada no aparelho (configurável pelo usuário em ajustes de "saída de conversão de texto em voz"); algumas vozes de maior qualidade podem usar rede. Para o checkpoint de privacidade, teste em **modo avião**: se falar, é local.

Atenção! O erro clássico é chamar `speak()` logo após criar o objeto — o `onInit` ainda não rodou e a chamada falha silenciosamente. Sempre condicione a fala ao estado `ready` (ou enfileire os textos até a engine inicializar).

12.13.4.4 STT on-device com `Whisper tiny` ([whisper.cpp](#))

Para garantir STT 100% local e independente do serviço do aparelho, o caminho mais usado é o [whisper.cpp](#) — implementação em C++ do Whisper, com bindings JNI e um exemplo Android oficial no repositório

(`examples/whisper.android`). O modelo `ggml-tiny` tem ~75 MB, e versões quantizadas ficam bem menores (técnicas do 1.6).

O fluxo tem três partes — capturar PCM, carregar o modelo, transcrever fora da main thread:

- **Código 1.8-04** — 1) Captura: PCM 16 kHz, mono, 16 bits — o formato que o Whisper espera — código completo no notebook companion (seção 1.8.4.4).
- **Código 1.8-05** — 2) e 3) Carga do modelo e transcrição — API do binding do exemplo oficial — código completo no notebook companion (seção 1.8.4.4).

Nota 20: o Whisper trabalha em **lote**: feche a janela de áudio (fim do push-to-talk ou VAD detectando silêncio) e então transcreva. Para comandos de poucos segundos, a latência do *tiny* quantizado em celulares recentes geralmente fica dentro do aceitável para conversa.

12.13.4.5 TTS on-device com Piper

O **Piper** é um sistema em C++/ONNX sem biblioteca Android oficial própria; o caminho prático em Android é o runtime **sherpa-onnx** (projeto k2-fsa), que executa vozes VITS/Piper localmente e oferece API Kotlin:

- **Código 1.8-06** — depende da dependência `sherpa-onnx` e de uma voz pt-BR do Piper — código completo no notebook companion (seção 1.8.4.5).

Todo o processamento é local: o texto nunca sai do aparelho.

12.13.4.6 Resumo: on-device ou serviço?

Veja, na Tabela 17, a seguir, um resumo sobre as abordagens, locais de processamento e as implicações para os *checkpoints*.

Tabela 17 – Abordagens de STT e TTS quanto ao local de processamento e à validação técnica

Abordagem	Onde processa	Implicação para os checkpoints
<code>SpeechRecognizer</code> (padrão)	Serviço do aparelho — pode usar rede	Declarar como híbrido, salvo se forçar on-device

<code>SpeechRecognizer</code> on-device (API 31+) / <code>EXTRA_PREFER_OFFLINE</code>	Local (com pacote de idioma baixado)	OK para IA local; documente a configuração
<code>TextToSpeech</code> (engine Google)	Local com voz baixada; rede possível	Teste em modo avião e documente
Whisper tiny via <code>whisper.cpp</code>	100% local	Atende IA local e privacidade
Piper via <code>sherpa-onnx</code>	100% local	Atende IA local e privacidade

Fonte: autoria própria.

Na prática: para a demo do hackathon, comece com **push-to-talk + APIs nativas** — você terá o ciclo falar → transcrever → responder → ouvir funcionando em uma tarde. Depois, troque as peças nativas por Whisper tiny e Piper para cravar o requisito de IA local, e por fim adicione VAD/wake word se a experiência mãos-livres for o diferencial do seu projeto.

12.13.5 Resumo / cheatsheet

- **STT** = fala → texto (entender o usuário); **TTS** = texto → fala (responder). Nos óculos sem display, voz é entrada e saída.
- Pipeline STT: áudio PCM → **mel-spectrogram** → modelo acústico/decodificador → texto; end-to-end faz tudo em uma rede (CTC, atenção, RNN-T).
- Pipeline TTS: texto → normalização → fonemas → modelo acústico (mel) → **vocoder** (onda); VITS/Piper são end-to-end.
- On-device viáveis: **Whisper tiny/base quantizado** e **Vosk** (STT); **Piper** (TTS). Grandes (Whisper large, Tacotron+WaveNet) não cabem no celular.
- Cascata de ativação: **wake word** (minúscula, sempre-ligada) → **VAD** (filtra silêncio) → STT (pesado, sob demanda). **Diarização** = "quem falou quando", para áudio multi-locutor.
- "Hey Meta" **não é acessível** pelo toolkit — wake word própria roda no celular, sobre o áudio vindo dos óculos (tópico 2.6).
- `SpeechRecognizer` padrão pode usar rede; para local: `EXTRA_PREFER_OFFLINE` / `createOnDeviceSpeechRecognizer` (API 31+). `TextToSpeech` inicializa **assíncrono** — nunca `speak()` antes do `onInit`.

- Inferência de voz sempre fora da main thread (coroutines, 1.2); escuta contínua cobra bateria (1.5); quantização e formatos são o 1.6.

12.13.6 Referências

- SPEECHRECOGNIZER / ANDROID DEVELOPERS. Disponível em: <https://developer.android.com/reference/android/speech/SpeechRecognizer>. Referência completa da API de reconhecimento, incluindo `createOnDeviceSpeechRecognizer` e códigos de erro.
- RECOGNIZERINTENT / ANDROID DEVELOPERS. Disponível em: <https://developer.android.com/reference/android/speech/RecognizerIntent>. Extras de configuração do reconhecimento (`EXTRA_LANGUAGE`, `EXTRA_PREFER_OFFLINE`, `EXTRA_PARTIAL_RESULTS`).
- TEXTTOSPEECH / ANDROID DEVELOPERS. Disponível em: <https://developer.android.com/reference/android/speech/tts/TextToSpeech>. Referência da API de síntese: inicialização, filas, `UtteranceProgressListener`.
- WHISPER / OPENAI (GITHUB). Disponível em: <https://github.com/openai/whisper>. Modelo, variantes (tiny a large), idiomas suportados e artigos associados.
- WHISPER.CPP / GGGERGANOV (GITHUB). Disponível em: <https://github.com/ggml-org/whisper.cpp>. Implementação C++ do Whisper com modelos ggml quantizados e exemplo Android.
- PIPER / RHASSPY (GITHUB). Disponível em: <https://github.com/rhasspy/piper>. TTS neural local baseado em VITS, com catálogo de vozes por idioma.
- SILERO VAD / SNAKERS4 (GITHUB). Disponível em: <https://github.com/snakers4/silero-vad>. Detector neural de atividade de voz, leve e pronto para uso.

- VOSK API / ALPHACEP (GITHUB). Disponível em:
<https://github.com/alphacep/vosk-api>. Toolkit de ASR offline com suporte a Android e modelos compactos por idioma.

12.13.6.1 Para ir além



SHERPA-ONNX / K2-FSA (GITHUB). Disponível em:
<https://github.com/k2-fsa/sherpa-onnx>. Runtime de fala (STT, TTS, VAD) com exemplos Android — o caminho prático para vozes Piper no celular.



OPENWAKEWORD / DSCRIPKA (GITHUB). Disponível em:
<https://github.com/dscripka/openWakeWord>. Framework open source de wake word para construir a frase de ativação da sua equipe.



FAIRSEQ (WAV2VEC 2.0) / META AI (GITHUB). Disponível em:
<https://github.com/facebookresearch/fairseq>. Código e modelos oficiais do Wav2Vec 2.0 para quem quiser explorar fine-tuning.



 **CEIA** |  **Meta**